

6. 运算方法和运算器

2026 春

杨莉萍
信息学院大数据系

本章主要内容

- 补码加/减法运算
- 定点数的移位运算
- 定点乘法运算
- 定点除法运算
- 浮点数运算
- 运算器组织。

定点加法、减法运算

谭教材3.2节

目录：

补码加减法

溢出概念与检测方法

基本的二进制加法/减法器。

■ 补码加法

$$\text{公式: } [x+y]_{\text{补}} = [x]_{\text{补}} + [y]_{\text{补}}$$

补码减法

公式:

$$[x - y]_{\text{补}} = [x]_{\text{补}} + [-y]_{\text{补}}$$

$$[-y]_{\text{补}} = \neg[y]_{\text{补}} + 2^{-n} \text{ 即对 } [y]_{\text{补}} \text{ 进行求补操作。}$$

练习(1) 二进制 $x=1001$ ， $y=-0101$ ，求 $x+y=?$

解： $[x]_{\text{补}} = 01001$ ， $[y]_{\text{补}} = \dots\dots$

$$\begin{array}{r} [x]_{\text{补}} \quad 0 \ 1 \ 0 \ 0 \ 1 \\ + \quad [y]_{\text{补}} \quad [\text{填空1}] \\ \hline \end{array}$$

$\therefore x+y = [\text{填空3}]$ (填写真值) $[x+y]_{\text{补}} [\text{填空2}]$ (符号位的进位舍去)

练习(2) $x=1101$ ， $y=0110$ ，求 $x-y=?$

解： $[x]_{\text{补}} = 01101$

$[y]_{\text{补}} = 00110$ ， $[-y]_{\text{补}} = \dots\dots$

$$\begin{array}{r} [x]_{\text{补}} \quad 0 \ 1 \ 1 \ 0 \ 1 \\ + \quad [-y]_{\text{补}} \quad [\text{填空4}] \\ \hline \end{array}$$

$[x-y]_{\text{补}} [\text{填空5}]$

$\therefore x-y = \dots\dots$ 。

作答

■ 溢出的概念

□ 可能产生溢出的情况——

- ◆ 两正数相加，结果变负数：正溢 (大于机器所能表示的最大数)
- ◆ 两负数相加，变正数：负溢 (小于机器所能表示的最小数)

下面举例子。

溢出概念与检测方法

[例1] $x = +1011$, $y = +1001$, 求 $x+y$ 。

解: $[x]_{\text{补}} = 01011$, $[y]_{\text{补}} = 01001$

$$\begin{array}{r} \phantom{[x]_{\text{补}}} 1 0 1 1 \\ + \phantom{[x]_{\text{补}}} 1 0 0 1 \\ \hline [x+y]_{\text{补}} 1 0 0 0 \end{array}$$

两个正数相加的结果成为负数，表示正溢。

溢出概念与检测方法

[例2] $x = -1101$, $y = -1011$, 求 $x+y$ 。

解: $[x]_{\text{补}} = 10011$, $[y]_{\text{补}} = 10101$

$$\begin{array}{r} \phantom{[x]_{\text{补}}} \phantom{[y]_{\text{补}}} 0 0 1 1 \\ + \phantom{[x]_{\text{补}}} [y]_{\text{补}} 0 1 0 1 \\ \hline [x+y]_{\text{补}} \color{red}{0} 1 0 0 0 \end{array}$$

两个负数相加的结果成为正数，表示负溢。

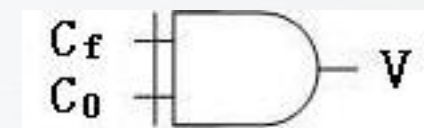
单符号位法:

■ C_f C_0

0	0	正确
0	1	正溢 (如前面例1)
1	0	负溢 (如前面例2)
1	1	正确 (如前面练习(1))

C_f 为符号位产生的进位, C_0 为数据位的最高有效位产生的进位。

异或电路



■ $V = C_f \oplus C_0$, 其中 C_f 为符号位产生的进位, C_0 为数据位的最高有效位产生的进位。 V 为1时, 表示溢出; 否则, 无溢出。

双符号位(变形补码)溢出检测法:

$$\begin{array}{r} 00.10101 \\ + 00.01000 \\ \hline 00.11101 \end{array}$$

正常 结果

$$\begin{array}{r} 00.10101 \\ + 00.11000 \\ \hline 01.01101 \end{array}$$

非正常符号位, 溢出(正溢)

$$\begin{array}{r} 11.10101 \\ + 11.11000 \\ \hline 11.01101 \end{array}$$

符号位进位 舍去, 正常结果

$$\begin{array}{r} 11.00101 \\ + 11.11000 \\ \hline 11.1101 \end{array}$$

非正常符号位, 溢出

- 双符号位中的最高符号位 永远是正确符号位
- 此时, 判溢出的表达式: $V = S_{f1} \oplus S_{f2}$

- 数据在运算中可能会产生溢出，运算器必须有相应的电路检测溢出。
- C语言程序设计中运算产生了溢出会带来什么问题，作为程序员你是否考虑了溢出问题？
- 若不处理这种溢出会产生什么问题：美国阿里安娜5火箭就是因为长数据向短数据转换过程中发生了运算溢出，直接导致系统崩溃，火箭爆炸，造成了巨大损失：“一行代码‘葬送’了 5 亿欧元；史上最贵 Bug”。
- **同学们需关注工程与社会，具备工程师的责任感！**

|| (多位)二进制加法运算

- 各位逐位相加，进位从右至左传递
- $\therefore$ 首先要考虑一位加法，然后考虑进位链。

$$\begin{array}{r} X_{n-1} \dots\dots\dots X_2 \ X_1 \ X_0 \\ + \quad Y_{n-1} \dots\dots\dots Y_2 \ Y_1 \ Y_0 \\ \hline ?_{n-1} \dots\dots\dots ?_2 \ ?_1 \ ?_0 \end{array}$$

一位加法逻辑电路实现

■ 规则:

■ $0 + 1 = 1$

■ $1 + 0 = 1$

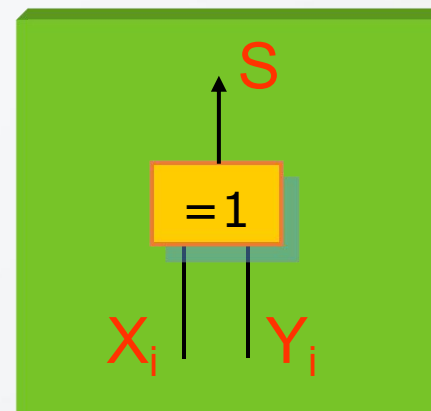
■ $0 + 0 = 0$

■ $1 + 1 = 0$

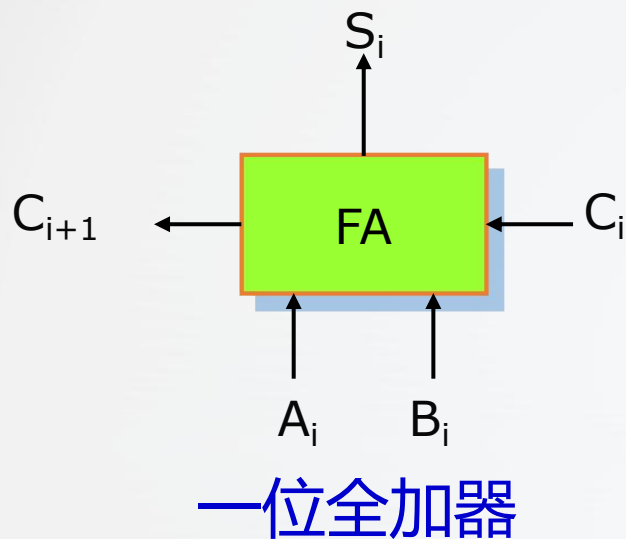
■ 一个异或门 即可实现自动一位加法

■ 算术运算变成逻辑电路:

$$S = X \oplus Y$$



带进位链 的 **一位** 全加器



被加数 A_i	加数 B_i	低位进位 C_i	和 S_i	进位 C_{i+1}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

$$S_i = A_i \oplus B_i \oplus C_i$$

$$C_{i+1} = A_i B_i + A_i C_i + B_i C_i \quad (1)$$

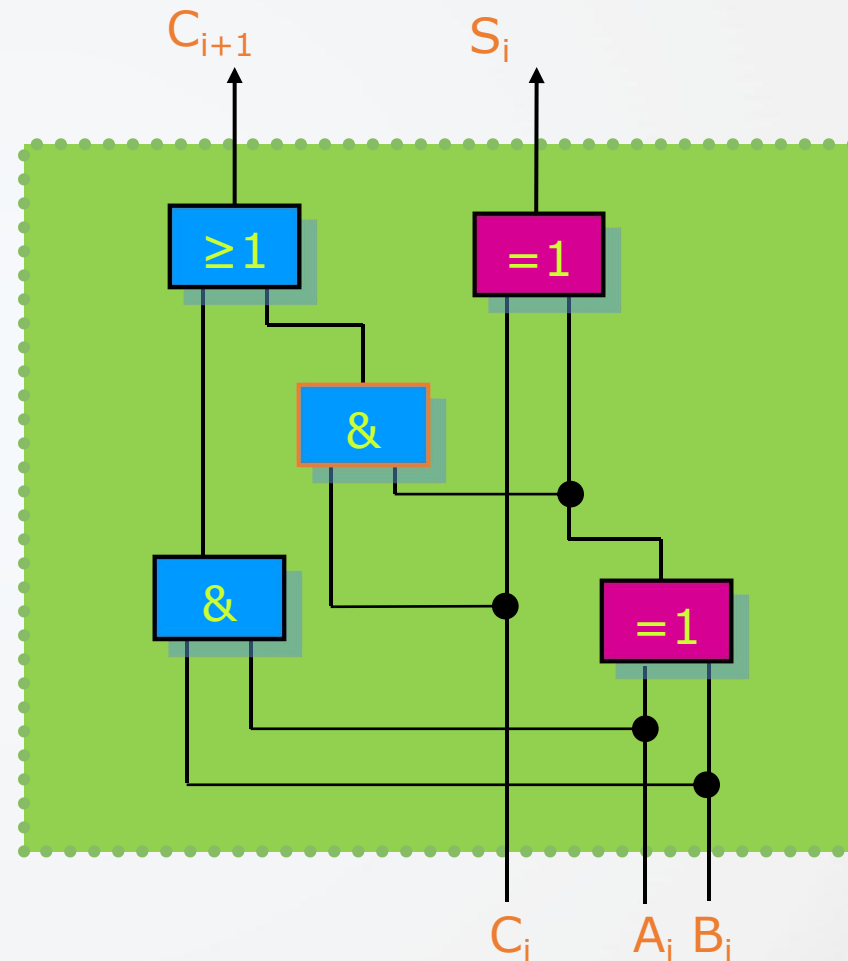
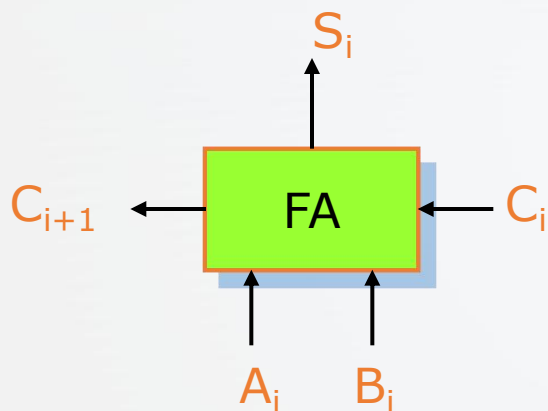
对**(1)**变换可得:

$$C_{i+1} = A_i B_i + (A_i \oplus B_i) C_i$$

一位全加器内部的逻辑实现

$$S_i = A_i \oplus B_i \oplus C_i$$

$$C_{i+1} = A_i B_i + (A_i \oplus B_i) C_i$$

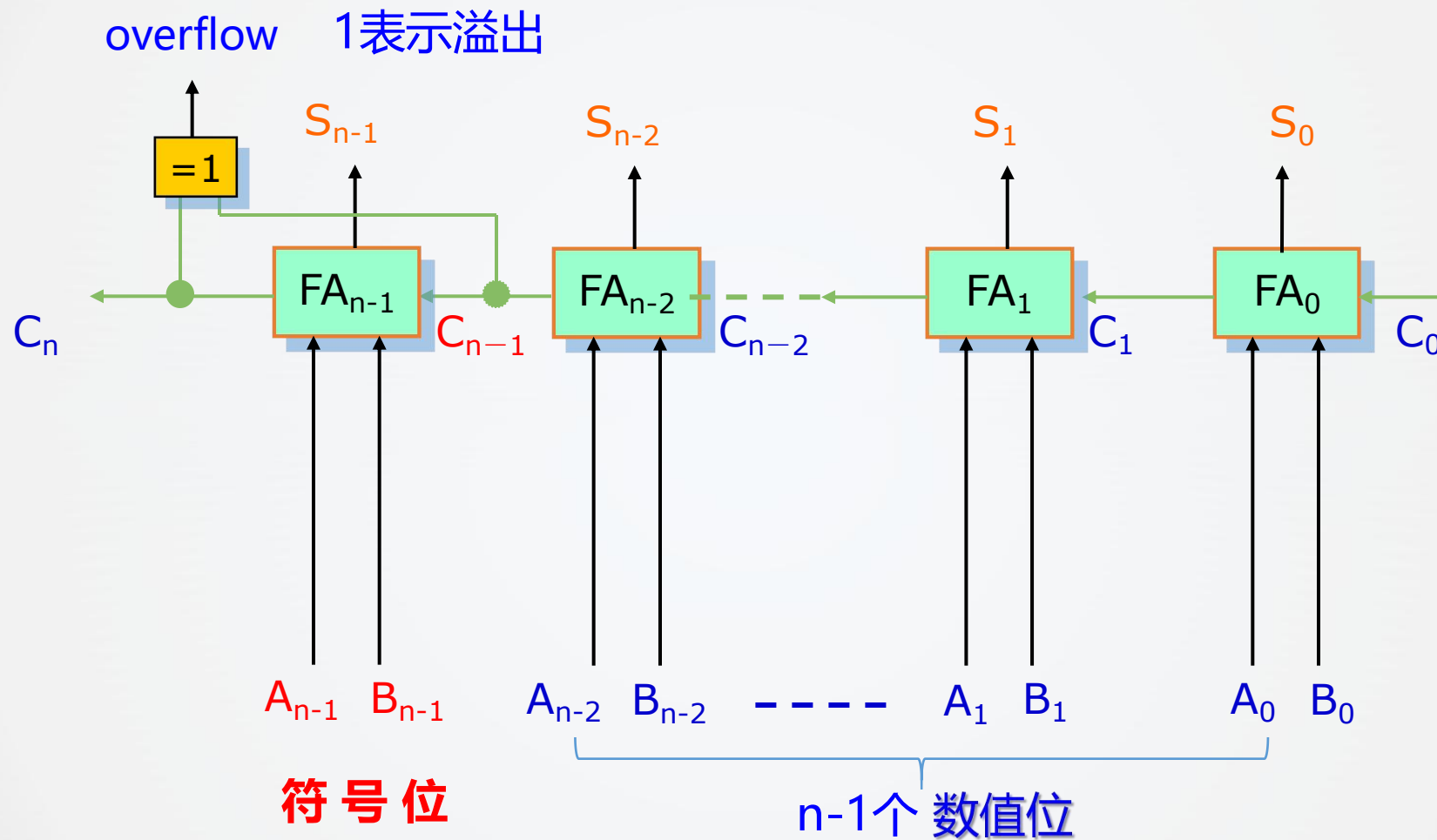


|| 多位加法器 (※)

实现的思路：

- n 位加法器包含 n 个一位全加器
- 将多个 一位全加器 串联
- 低位进位输出 连接到高位进位输入。

单符号位补码 加法器电路



补码 减法电路实现 (难)

补码减法可以转换为加法

$$[X - Y]_{\text{补}} = [X]_{\text{补}} + [-Y]_{\text{补}}$$

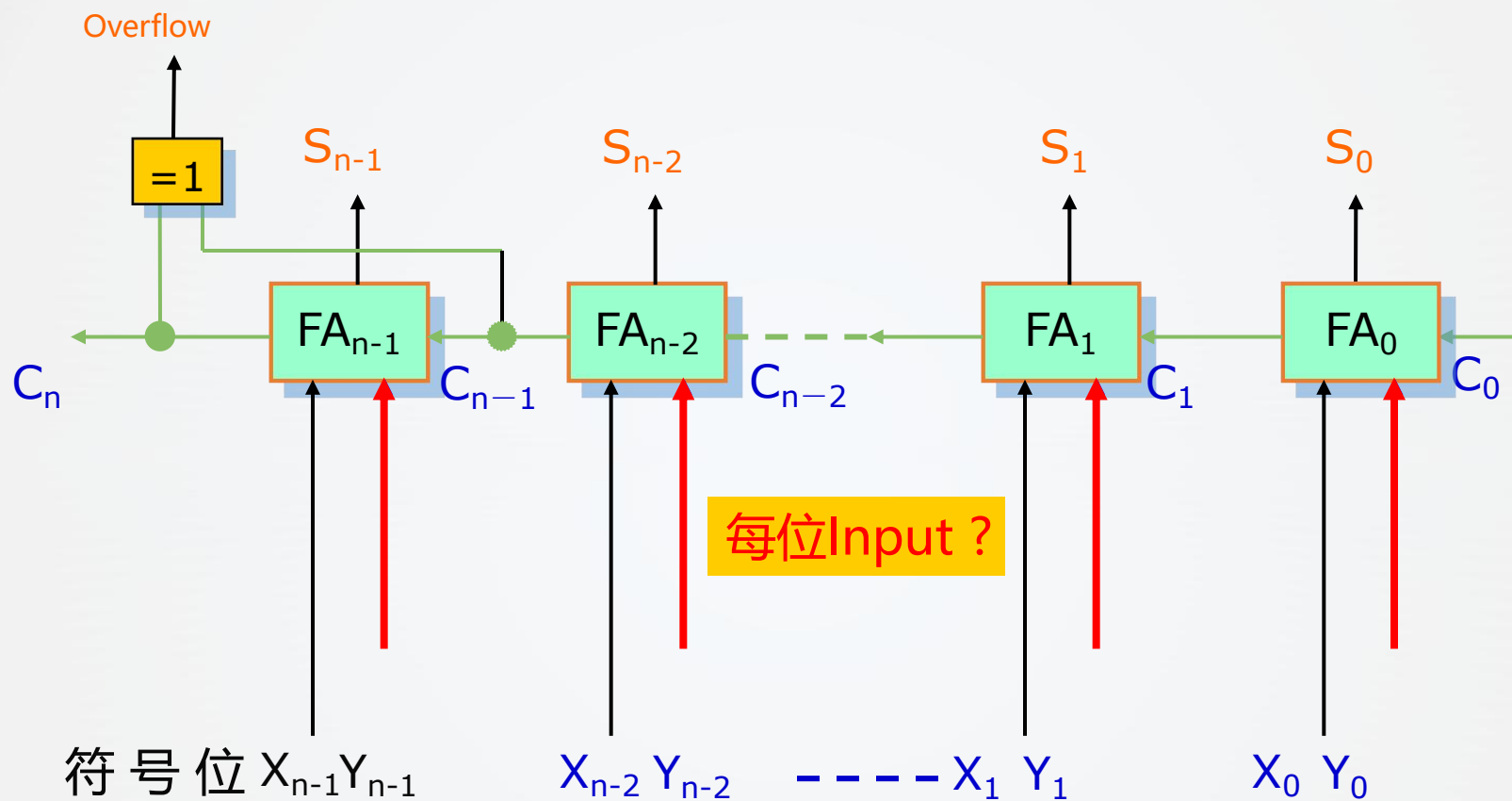
实现减法的关键是 求 $-Y$ 的补码——

方法：将 $Y_{\text{补}}$ 连同符号位一起，逐位取反末位加一

$$[-Y]_{\text{补}} = [[Y]_{\text{补}}]_{\text{补}}$$

$$\text{差 } S = [X]_{\text{补}} + [-Y]_{\text{补}}。$$

加法器的改造



加法器：输入 $[Y]_{\text{补}}$ 做加法， 如果输入 $[-Y]_{\text{补}}$ 则做减法。

减法时: $[-Y]_{\text{补}} = [[Y]_{\text{补}}]_{\text{补}} = [[Y]_{\text{补}}]_{\text{反}} + 1$

■ 引入 **Sub** 位:

□ **Sub=0** 表示做加法, 送入加法器的应是 $[Y]_{\text{补}}$ (的每位)

□ **Sub=1** 表示做减法, 送入加法器的应是 $[-Y]_{\text{补}}$ 。如下:

$[Y]_{\text{补}}$ 每位

Y_i	Sub	Input
Y_i	0	Y_i
Y_i	1	$\overline{Y_i}$

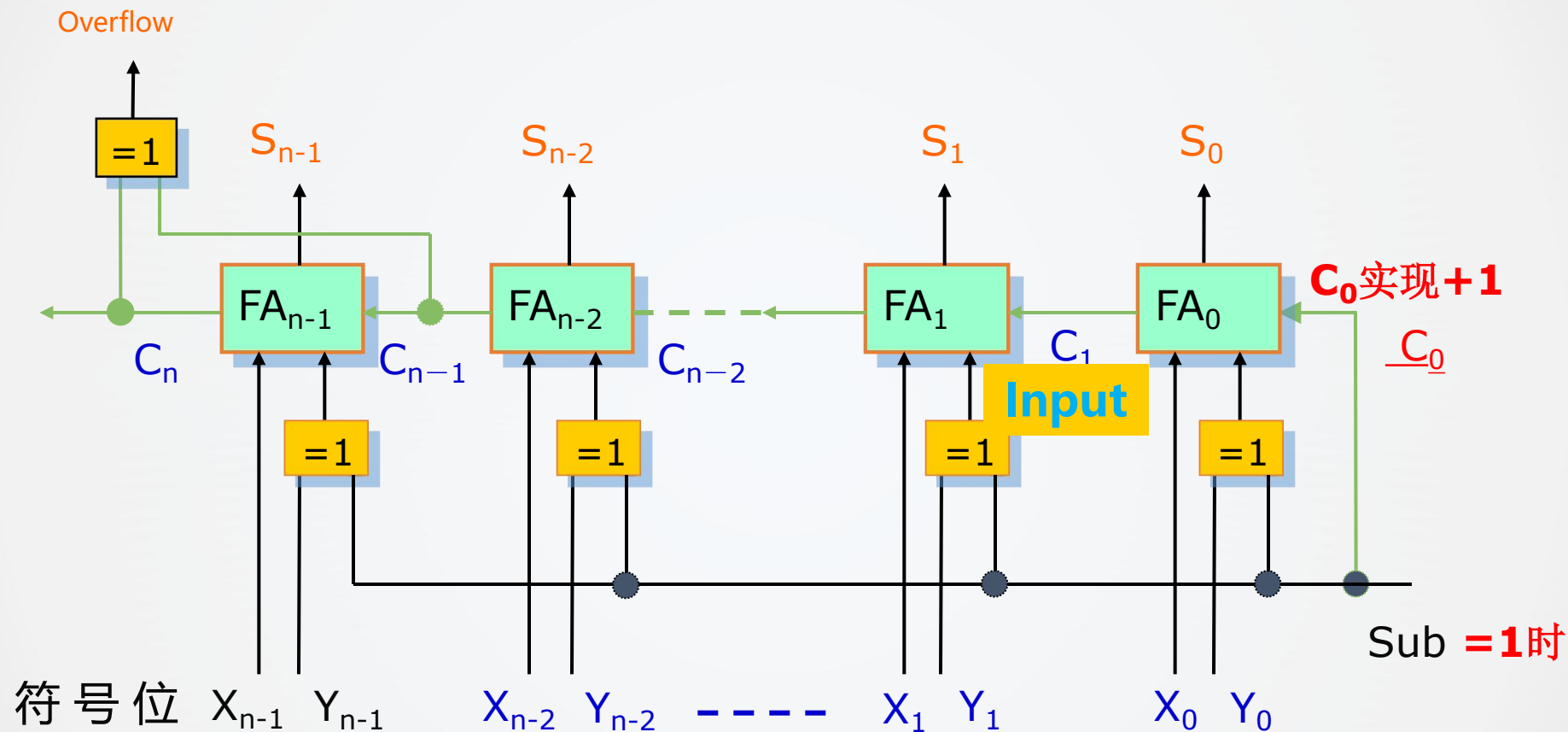
• 正好利用异或性质: 对任意二进制1位 a

$$a \oplus 0 = a$$

$$a \oplus 1 = \overline{a}$$

• 因此: $Input = Y_i \oplus Sub$ 。

单符号补码加/减法器 电路实现



引入sub的减法，减少了逻辑器件。

- 移位是乘法实现过程中的基本操作
- 移位操作包括 逻辑移位(被移位数据是逻辑量 无符号)和 算术移位(带符号(位)的); 每种移位操作又可分为左移和右移.
- 见下表, 以补码为例:

表 3.1 移位操作的特点				
移位类型		移位前的数据	移位后的数据	特 点
逻辑移位	左移	11101110	11011100	左边一位移出, 右边空出位补 0
	右移	11101110	01110111	右边一位移出, 左边空出位补 0
算术移位	左移	11101110	11011100	各位依次左移, 右边空出位补 0, 一次移位相当于乘以 2, 当符号位改变时表明溢出
	右移	11101110	11110111	符号位保持不变, 其余各位依次右移, 最右边一位移出, 将符号位复制到左边空出的位, 一次移位相当于除以 2

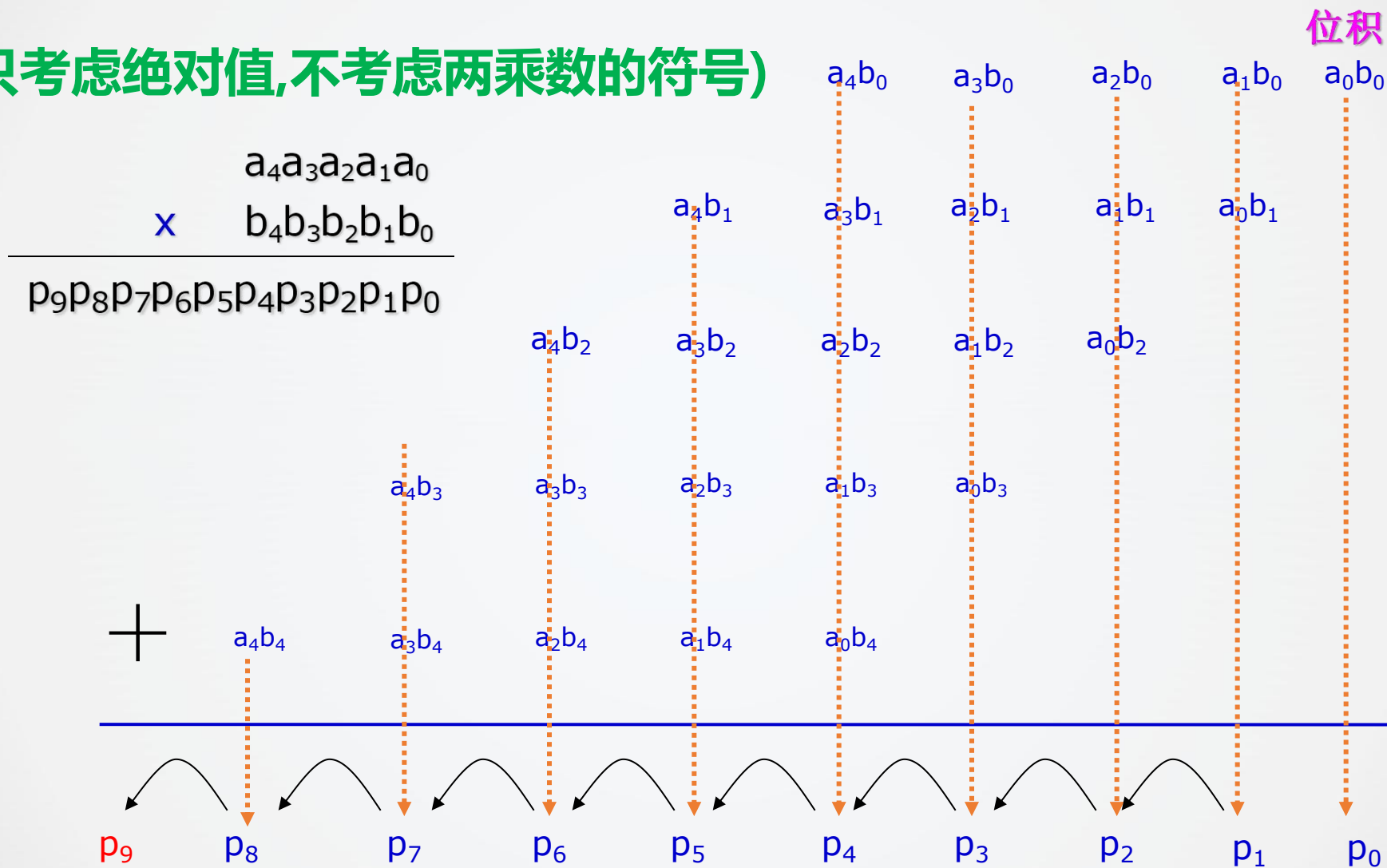


目录:

- 1、原码一位乘法
- 2、补码一位乘法
- 3、阵列乘法器。

|(人工) 二进制多位乘法运算

(先只考虑绝对值,不考虑两乘数的符号)



先计算相加数(位积), 然后逐列相加 (注意进位)。

原理：

- 符号位单独参加运算，数据位取绝对值参加运算。
- 运算法则：

设： $[X]_{\text{原}} = X_0 \cdot X_1 X_2 \dots X_n$ $[Y]_{\text{原}} = Y_0 \cdot Y_1 Y_2 \dots Y_n$

则： $P_0 = \underline{X_0 \oplus Y_0}$ $|P| = |X| \cdot |Y|$

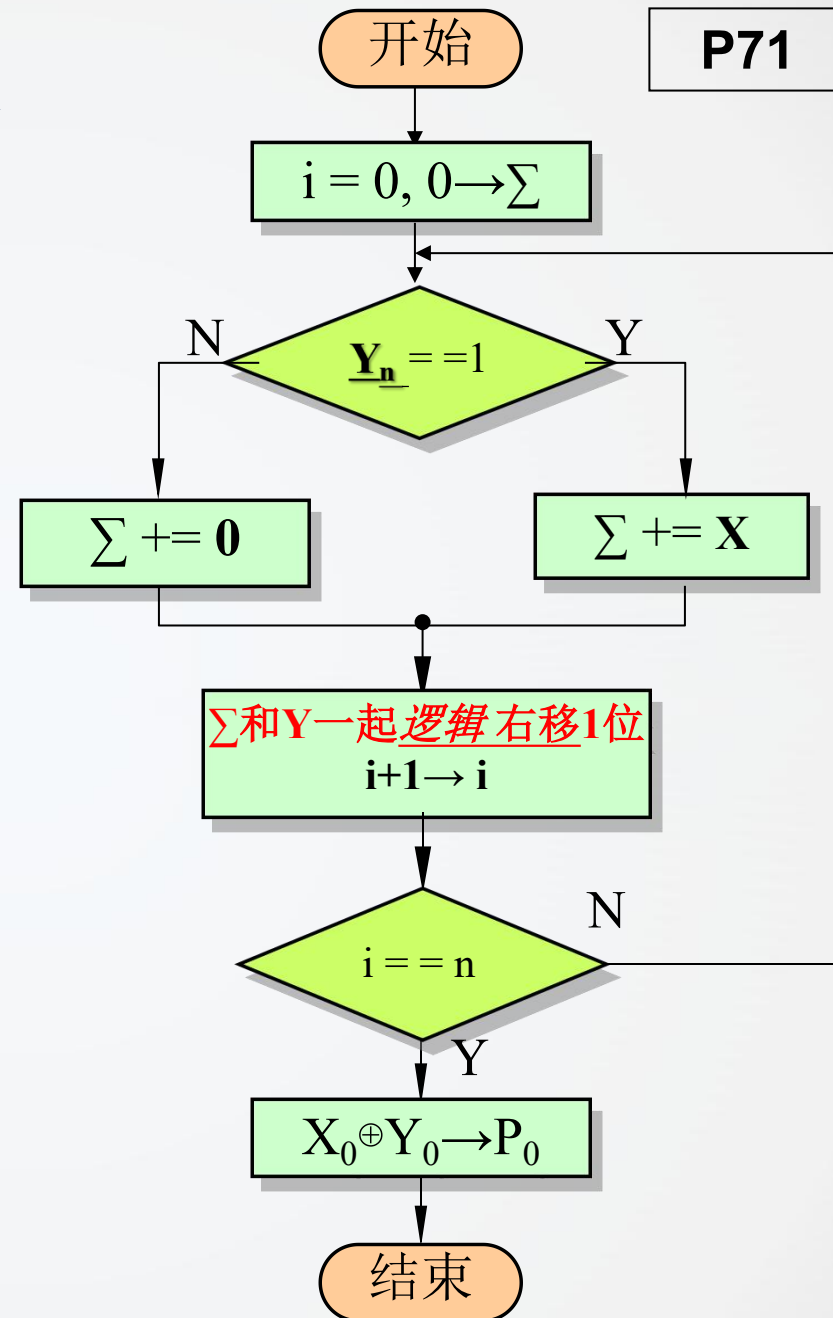
- 设 $|x| = 1101$, $|y| = 1011$

					1	1	0	1	(x)
x					1	0	1	1	(y)
					1	1	0	1	
			1		1	0	1		
		0	0		0	0			
+		1	1	0	1				
	1	0	0	0	1	1	1	1	(z)

➤ 乘积的最后是所有部分积求和，普通的加法器无法一次实现。

原码一位乘法的机器算法 对人工手算方法作如下修改:

- (1) 设**初始 部分积 Σ 为0**，每得到一个**部分积**就与前面的部分积做一次加法运算，即：以累加代替所有部分积同时相加。
- 这样减少了保存部分积所需的寄存器数目，且避免了一次性部分积求和的不便。
- (2) 用**部分积 右移**代替手算中的**位积左移**：这样不仅使部分积相加运算可固定在同一位置上进行，且不再需要长度为 $2n$ 位的加法器(可**利用乘数的 n 位寄存器**)。



原码乘法算法流程图

例3.10:

P71

- 已知 $X=0.1101$, $Y= - 0.1011$
- 用原码一位乘法计算 $X \times Y$ 。

$$|X| = 00.1101,$$

$$|Y| = 0.1011$$

部分积 R_0 乘数 R_1 Y_n 判断位

0	0	0	0	0	0	0	1	0	1	<u>1</u>
---	---	---	---	---	---	---	---	---	---	----------

+ |X| 整数部分的0

0	0	1	1	0	1					
---	---	---	---	---	---	--	--	--	--	--

→

0	0	0	1	1	0	1	0	1	0	<u>1</u>
---	---	---	---	---	---	---	---	---	---	----------

+ |X|

0	0	1	1	0	1					
---	---	---	---	---	---	--	--	--	--	--

<u>0</u>	<u>1</u>	0	0	1	1					
----------	----------	---	---	---	---	--	--	--	--	--

→

0	<u>0</u>	<u>1</u>	0	0	1	1	1	0	1	<u>0</u>
---	----------	----------	---	---	---	---	---	---	---	----------

+ 0

0	0	0	0	0	0					
---	---	---	---	---	---	--	--	--	--	--

0	0	1	0	0	1					
---	---	---	---	---	---	--	--	--	--	--

→

0	0	0	1	0	0	1	1	1	0	<u>1</u>
---	---	---	---	---	---	---	---	---	---	----------

+ |X|

0	0	1	1	0	1					
---	---	---	---	---	---	--	--	--	--	--

0	1	0	0	0	1					
---	---	---	---	---	---	--	--	--	--	--

→

0	0	1	0	0	0	1	1	1	1	0
---	---	---	---	---	---	---	---	---	---	---

又 $\because$ 符号位 P_f
 $= X_0 \oplus Y_0$
 $= 0 \oplus 1 = 1$
 $\therefore X \times Y =$
 $- 0.10001111。$

绿色部分 是要取的(部分)积结果!

■ 这里算得的是乘积的补码。

■ 乘数 $[Y]_{\text{补}}$ 记作 $Y_0.Y_1Y_2\cdots Y_n$

■ 由 谭教材P73推得：

$[X \times Y]_{\text{补}}$

注：这项
不移位！

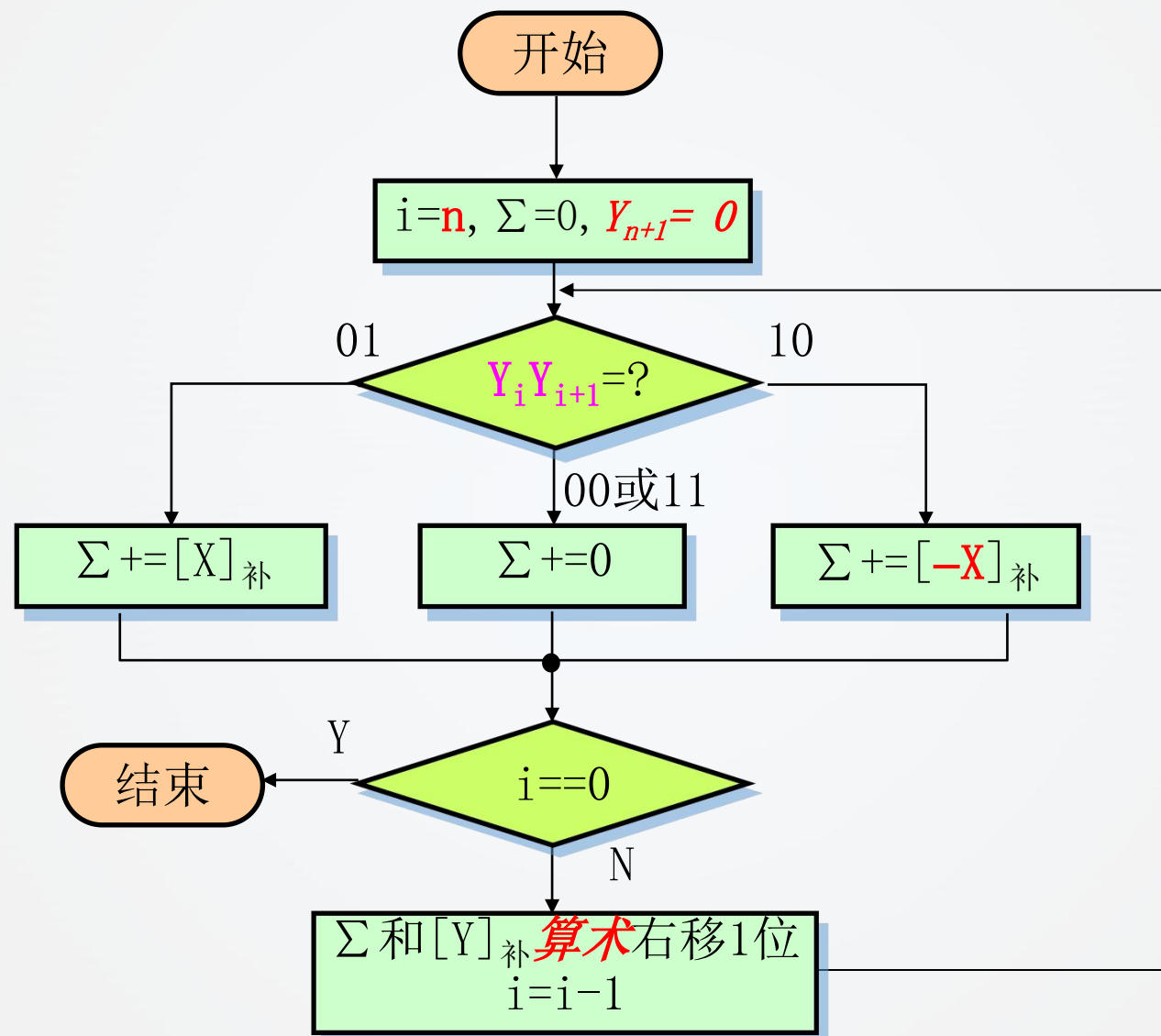
$$= [X]_{\text{补}} \times [(Y_1 - Y_0) + (Y_2 - Y_1)2^{-1} + (Y_3 - Y_2)2^{-2} + \cdots + (0 - Y_n)2^{-n}]$$

对比：

$$XY = X \times [Y_12^{-1} + Y_22^{-2} + \cdots + Y_n2^{-n}]$$

原码一位乘法

补码一位乘法流程图(booth一位乘法)



例子:

用补码一位乘法计算 $[X \times Y]_{\text{补}}$

$$X = -0.10111, Y = 0.10101$$

解:

$$[X]_{\text{补}} = 11.01001, \quad [-X]_{\text{补}} = 00.10111$$

$$[Y]_{\text{补}} = 0.10101$$

$[X]_{\text{补}} = 11.01001$

$[-X]_{\text{补}} = 00.10111$

$[Y]_{\text{补}} = 0.10101$

部分积 R_0

乘数 R_1

判断位 $Y_n Y_{n+1}$

	0	0	0	0	0	0	0.	1	0	1	0	1	<u>0</u>
$+ [-X]_{\text{补}}$	0	0	1	0	1	1							
	0	0	1	0	1	1							
$\rightarrow$	<u>0</u>	0	0	1	0	1	1	1	0.	1	0	1	<u>0</u>
$+ [X]_{\text{补}}$	1	1	0	1	0	0	1						
	<u>1</u>	1	1	0	1	0	0						
$\rightarrow$	<u>1</u>	1	1	1	0	1	0	0	1	0.	1	0	<u>1</u>
$+ [-X]_{\text{补}}$	0	0	1	0	1	1	1						
	0	0	1	0	0	0	1						
$\rightarrow$	0	0	0	1	0	0	0	1	0	1	0.	1	<u>1</u>
$+ [X]_{\text{补}}$	1	1	0	1	0	0	1						
	1	1	1	0	0	0	1	1	0	1	0.	1	<u>1</u>

$[X]_{\text{补}} = 11.01001$
 $[-X]_{\text{补}} = 00.10111$
 $[Y]_{\text{补}} = 0.10101$



注: 做到 $Y_0 Y_1$ 时, 只加, 不移位!

所以 $[X \times Y]_{\text{补}} = \underline{1}.10000\ 11101$ 。

- 在本子上完成, **需写出具体计算过程:**
- 1. 谭教材习题: 3.6 (1), 3.7(1)(2)
- 2. 试题书P11: 2.3计算题:1.(用算术移位做)。

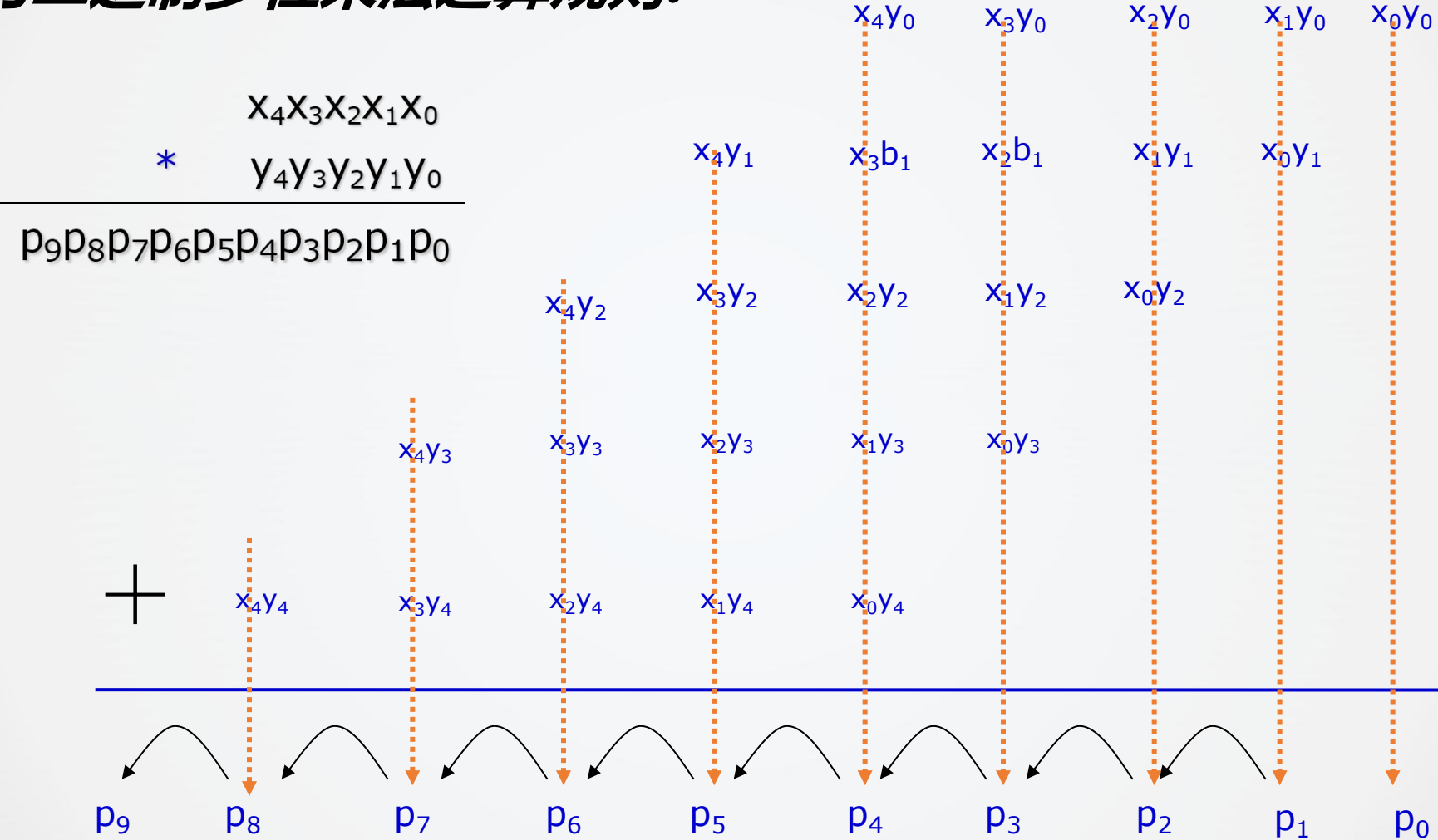


3、无符号数的阵列乘法器

- 之前介绍的原码、补码一位乘法主要是通过加法器的循环累加进行计算，速度较慢。为提高多个位积求和的速度，可以采用硬件的方式实现 阵列乘法器。

3、无符号数的阵列乘法器

复习二进制多位乘法运算规则:



先计算相加数(位积), 然后逐列相加 (注意进位)。

一位乘法的逻辑实现：

■ $R = X * Y$ 的运算规则：

□ $0 \times 0 = 0$

□ $0 \times 1 = 0$

□ $1 \times 0 = 0$

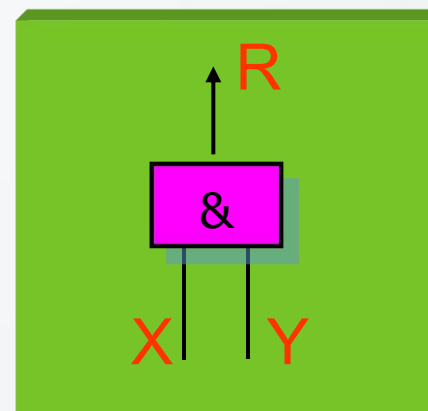
□ $1 \times 1 = 1$

算术乘

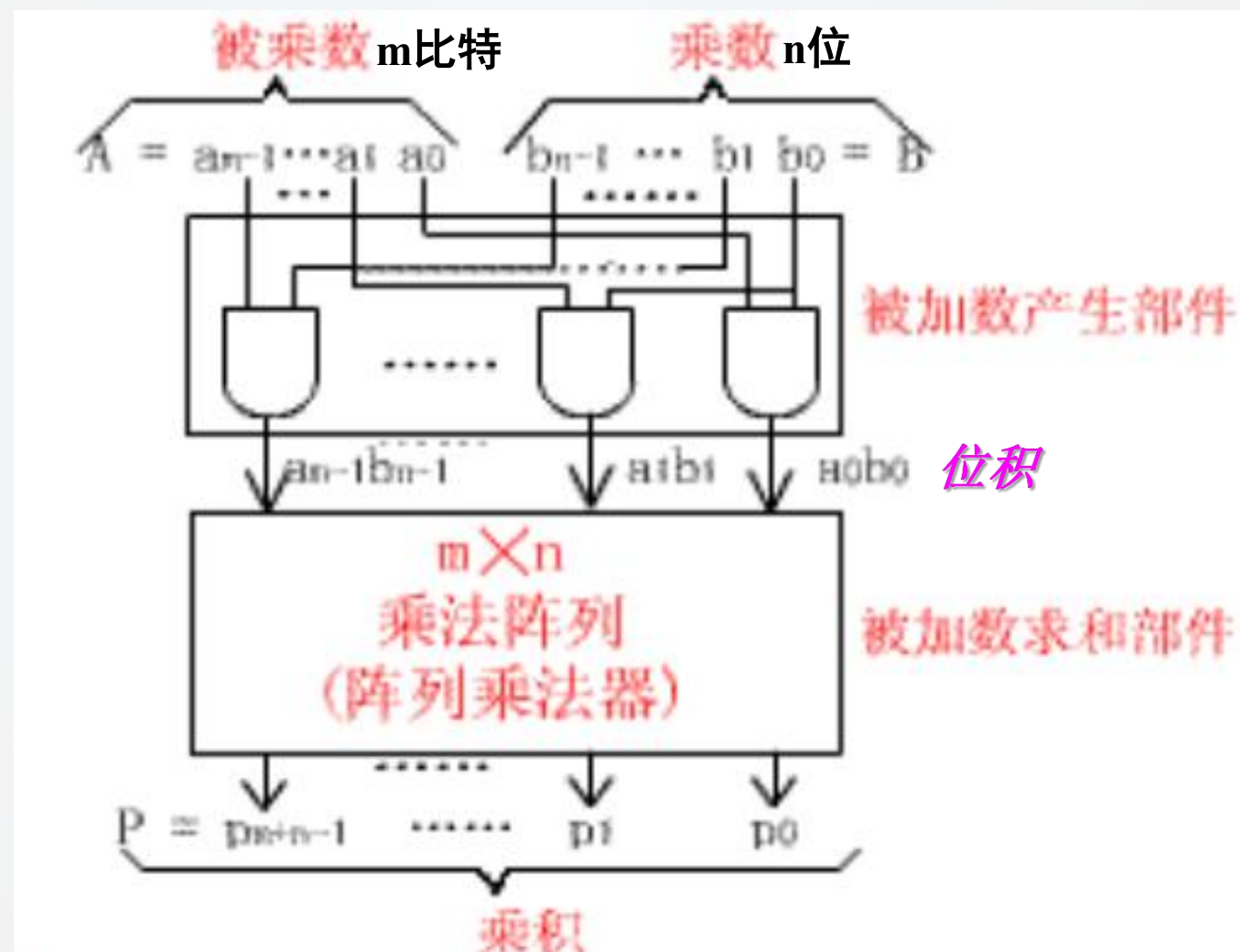
■ 一个与门即可实现一位乘法：

逻辑与

$$R = X \cdot Y$$



3、无符号数的阵列乘法器

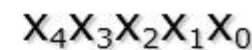


不带符号 阵列乘法器逻辑图

U



一位全加器 5个输入输出引脚的连接说明



*

$$Y_4 Y_3 Y_2 Y_1 Y_0$$
$$p_9 p_8 p_7 p_6 p_5 p_4 p_3 p_2 p_1 p_0$$

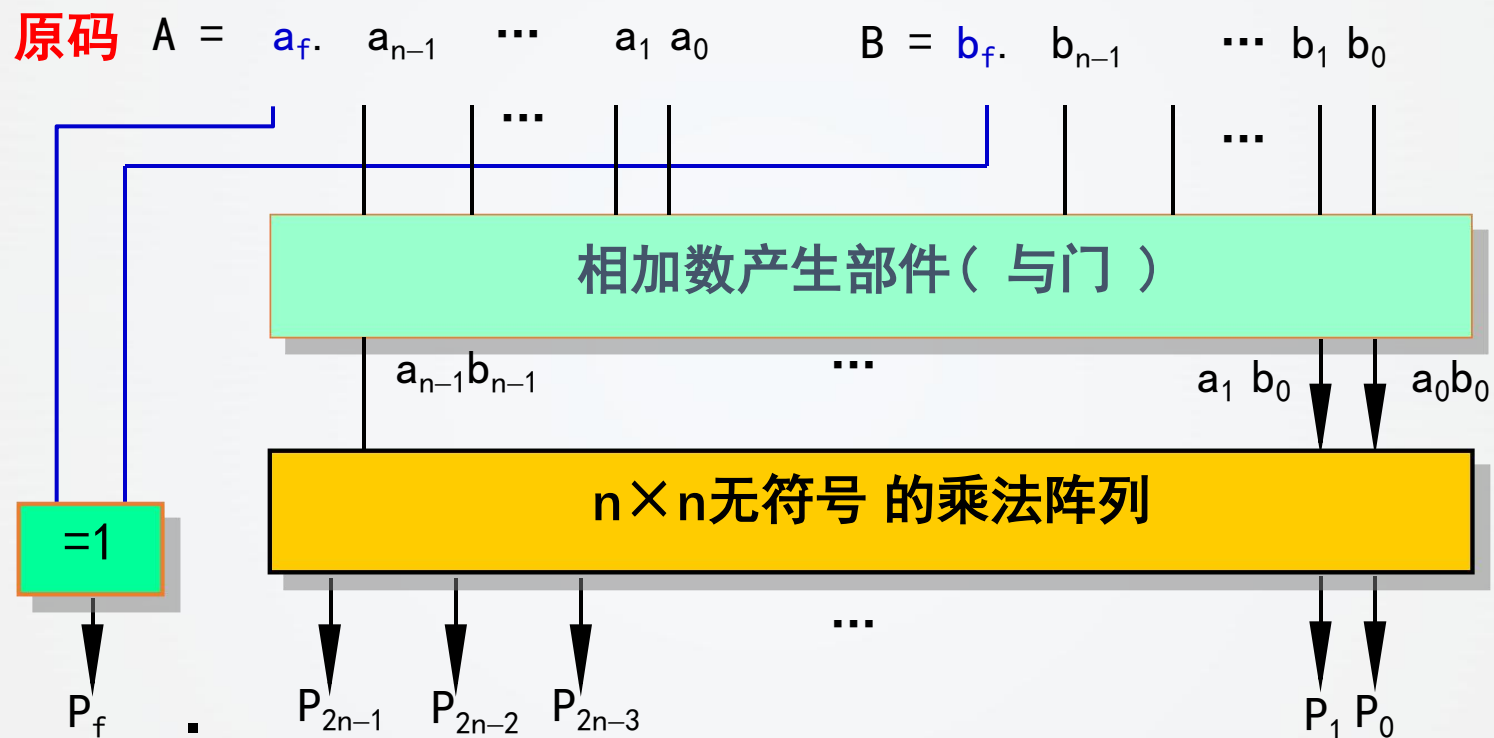

p8

p.

p

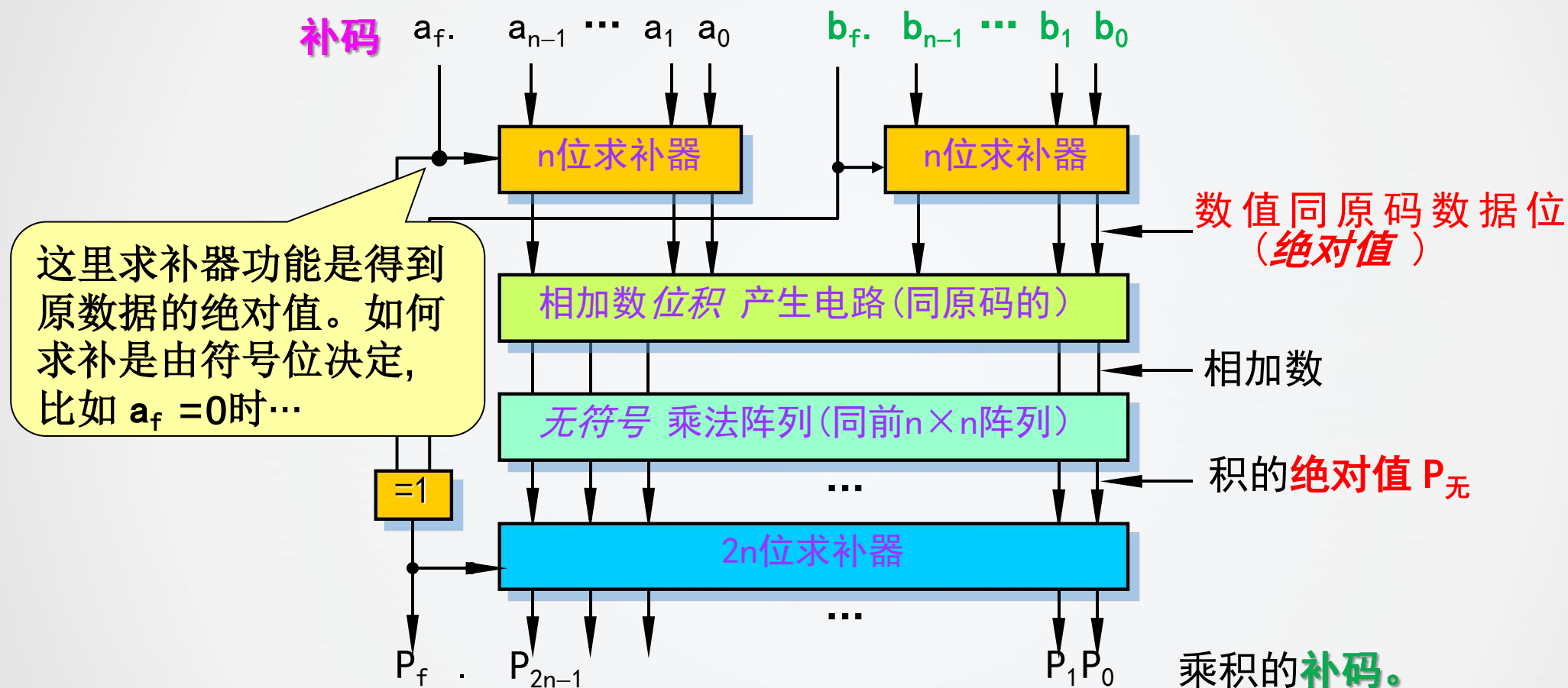
P

$n \times n$ 位原码阵列乘法器框图

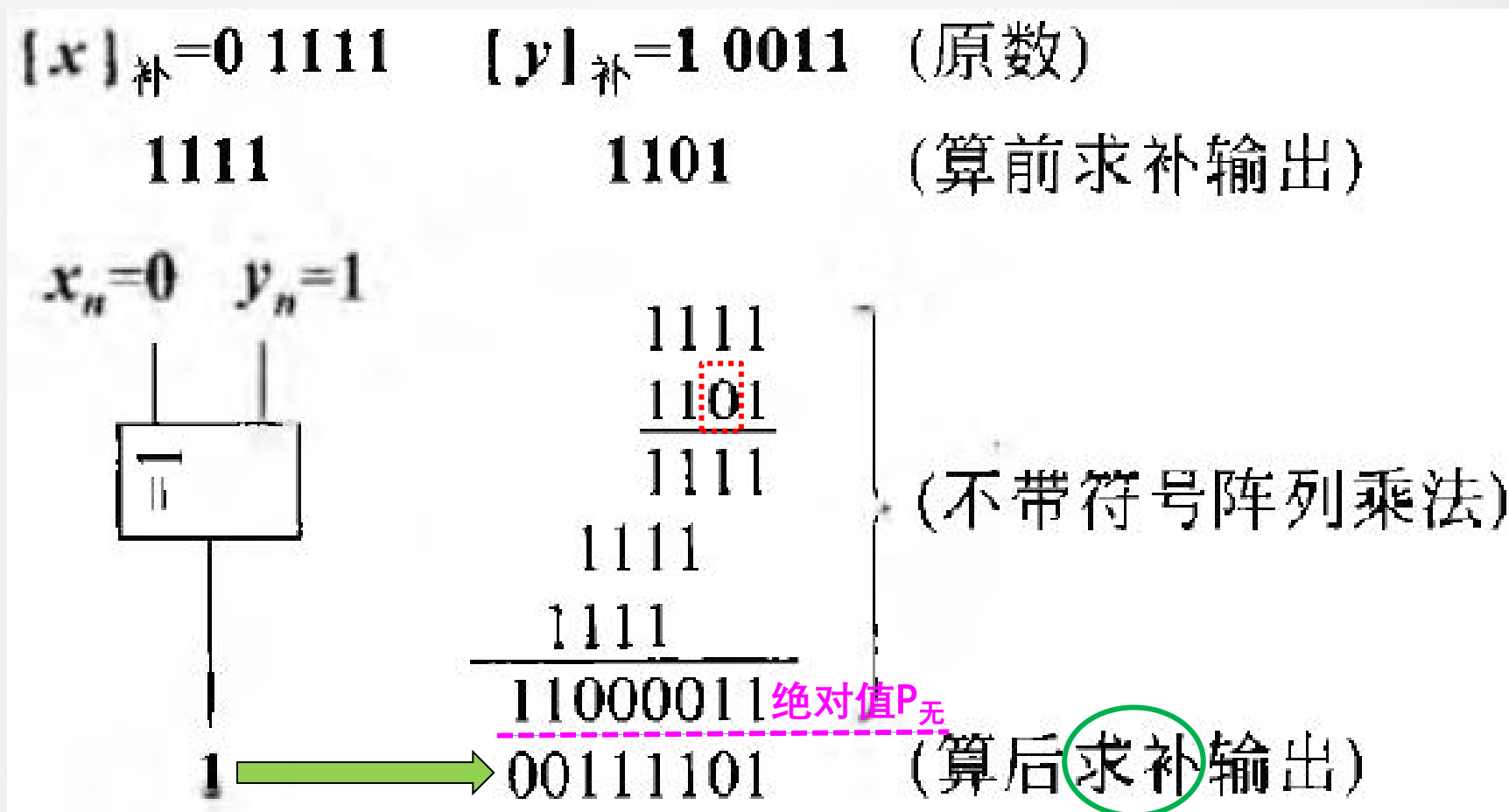


补码阵列乘法器原理图

- 以下利用无符号阵列乘法器, 实现补码乘法 $A_{补} * B_{补} = P_{补}$



补码阵列乘法器运算例子:



结果: $[X*Y]_{\text{补}} = 1\ 00111101$

3.4 定点除法运算

原码除法的算法原理:

设有n位定点小数（定点整数也适用）

被除数x, $[x]_{\text{原}} = x_f \cdot x_{n-1} \dots x_1 x_0$

除数y, $[y]_{\text{原}} = y_f \cdot y_{n-1} \dots y_1 y_0$

则有商 $q = x/y$,

$[q]_{\text{原}} = (x_f \oplus y_f) (0.x_{n-1} \dots x_1 x_0 / 0.y_{n-1} \dots y_1 y_0)$

商的符号运算 $q_f = x_f \oplus y_f$ 与原码乘法一样。

下页是人工算法例子。



定点一位除法实现方案(人工): x / y
 $0.1\ 0\ 0\ 1 / 0.1\ 0\ 1\ 1$

例 1: y

	0.1 1 1 0 1
0.1 0 1 1	0.1 0 0 1 0
-	0. 0 1 0 1 1
	0.0 0 1 1 1 0
-	0. 0 0 1 0 1 1
	0.0 0 0 0 1 1 0
	0. 0 0 0 1 0 1 1
	0.0 0 0 0 1 1 0 0
-	0. 0 0 0 0 1 0 1 1
	0.0 0 0 0 0 0 1

商 q	
$x(r_0)$	被除数
$2^{-1}y$	除数继续右移1位,减除数
r_1	得余数r1
$2^{-2}y$	除数继续右移1位,减除数
r_2	得余数r2
$2^{-3}y$	除数继续右移1位 试商, 不减 除数
r_3	得余数r3
$2^{-4}y$	除数继续右移1位,减除数
r_4	得余数r4

◆ 思考: 商上0还是商上1, 人可以比较后确定, 计算机如何确定?

- 得到的结果为原码 (加减计算过程中用补码).
- (1) 第1步,被除数减去除数Y ——试商
- (2) 当结果大于等于0时, 商上1; 否则, 转(3)。为获得下一位商需要 **先**将余数左移1位 (末位补0), **再**减Y 比较, 转(2).
- (3) 当结果小于0时, 商上0。此时,为获得下一位商需要将余数左移1位, **加Y** 比较 (原理见P79式(3-25)), 转(2)准备上商.
- (4) 商得到需要的精度时, 结束。

例1: $[X]_{\text{补}} = 00.1001$

被除数/除数

商 上商位 说明

$[Y]_{\text{补}} = 00.1011$
 $[-Y]_{\text{补}} = 11.0101$

$Q = \overline{R_0}$ 的符号位

00.1001		
+ $[-Y]_{\text{补}}$ 11.0101		
11.1110		0.
← 11.1100		
+ $[Y]_{\text{补}}$ 00.1011		
00.0111		0.1
← 00.1110		
+ $[-Y]_{\text{补}}$ 11.0101		
00.0011		0.11
← 00.0110		
+ $[-Y]_{\text{补}}$ 11.0101		
11.1011		0.110
← 11.0110		
+ $[Y]_{\text{补}}$ 00.1011		
00.0001		0.1101

减Y比较

$R_0 < 0$ 商Q上0
 左移一位, 加Y比较

$R_1 > 0$, 商上1
 左移一位, 减Y比较

$R_2 > 0$, 商上1
 左移一位, 减Y比较

$R_3 < 0$ 商上0
 左移一位, 加Y比较

$R_4 > 0$, 商上1

又 $\because$ 符号位 $Q_f = X_0 \oplus Y_0 = 0 \oplus 0 = 0$

$\therefore [X \div Y]_{\text{原}} = 0.1101$, 余数 $[r] = (0.0001)_2 \times 2^{-4}$

简单复习
浮点数表示
(教材P26-27)

1、浮点加减运算

设有两个浮点数 x 和 y ，它们分别为

$$x = 2^{E_x} \cdot M_x$$

$$y = 2^{E_y} \cdot M_y$$

$$\text{则 } x \pm y = M_x \cdot 2^{E_x} \pm M_y \cdot 2^{E_y}$$

其中 E_x 和 E_y 分别为数 x 和 y 的阶码， M_x 和 M_y 为数 x 和 y 的尾数。**两浮点数进行加减法的运算规则**是(不妨设 $E_x \leq E_y$)

$$x \pm y = (M_x \cdot 2^{\underline{E_x - E_y}} \pm M_y) \cdot 2^{E_y}。$$

2、浮点运算步骤如下(教材P83):

- 1) 比较阶码并完成对阶(**小阶向大阶对齐**);
- 2) 尾数求和(差)运算;
- 3) 结果规格化:可采用规格化形式——**绝对值0.1XXXX**
(也可采用IEEE754标准的, 详见P85)
- 4) **尾数的舍入处理**:末位恒置1 或 0舍1入
- 5) 阶码的溢出判断:详见教材P84。

尾数规格化的方法

■ 连同符号位一起移位

□ 01.XXXXX 右规 1位 → 00.1XXXX

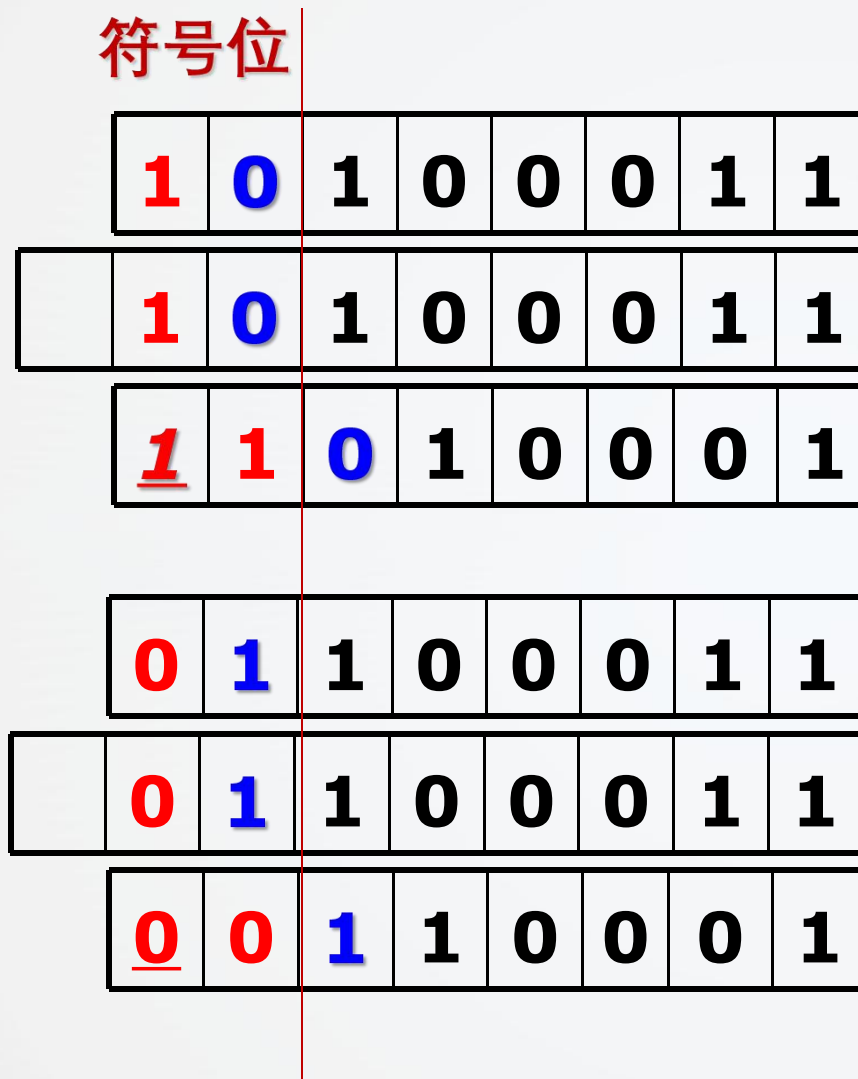
□ 10.XXXXX 右规 1位 → 11.0XXXX 成正常双符号位

(双符号位中的最高符号位 永远是正确符号位)

□ 正数 00.0XXXX 左规 直到 → 00.1XXX 0...0

□ 负数 11.1XXXX 左规 直到 → 11.0XXX 0...0。

向右规格化



尾数 连同符号位
数据位右移1位,
阶码加1

尾数连同符号位
数据位右移1位,
阶码加1。

符号位

0	0	0	0	1	0	0	1		
0	0	0	0	1	0	0	1	0	0

尾数左移两位,
阶码减2

1	1	1	0	0	0	0	1	
1	1	1	0	0	0	0	1	0

尾数左移一位,
阶码减1。

例1.两浮点数 $x = 2^{101} \times 0.11011011$, $y = 2^{111} \times (-0.10101100)$ 。
假设尾数在计算机中以补码表示, 可存储10位尾数, 含2位符号位(双符号位), 阶码以补码表示, 求 $x+y$ 。

解: 将 x, y 转换成浮点数据格式

$$[x]_{\text{浮}} = \underline{00} \ 101, 00.110110 \mathbf{11}$$

$$[Y]_{\text{浮}} = \mathbf{00 \ 111}, 11.01010100$$

步骤1: 对阶, 阶差 $E_x - E_y = [E_x]_{\text{补}} + [-E_y]_{\text{补}}$

$$[-E_y]_{\text{补}} = \mathbf{11 \ 001}$$

$$E_x - E_y = 00 \ 101 + 11 \ 001 = 11 \ 110 = -2 < 0$$

$E_x - E_y < 0$ 所以 $E_x < E_y$ 小阶向大阶看齐,

所以, x 阶码加2 x 尾数右移2位。

步骤2：尾数求和

$$\begin{aligned}[X+Y]_{\text{浮}} &= 00\ 111, \textcolor{red}{00}.00110110(\textcolor{green}{11}) \text{ 保留附加位(P83)} \\ &+ 00\ 111, 11.01010100 \\ &= 00\ 111, 11.\textcolor{red}{1}0001010(\textcolor{green}{11})\end{aligned}$$

步骤3：计算结果规格化

$[X+Y]_{\text{浮}}$ 为非规格化数, 尾数需左归1位, 阶码减1,

$$00\ \textcolor{red}{1}10, 11.\textcolor{green}{0}001010\ \textcolor{green}{1}(\textcolor{green}{1})$$

步骤4：舍入处理

$$[X+Y]_{\text{浮}} = \textcolor{red}{00}110, 11.\textcolor{green}{0}00101\textcolor{red}{10} \quad (\textcolor{red}{0}\text{舍}\textcolor{green}{1}\text{入法})$$

且结果已是规格化的。

步骤5：溢出判断

无溢出 $[X+Y]_{\text{浮}} = 2^{110} \times (-00.11101010)_2。$

例2 (特殊)

$$X=2^{111} \times 0.1111 \text{ 1111}, \quad Y=2^{111} \times 0.1000 \text{ 0001}$$

$$[X]_{\text{浮}} = 00 \text{ 111}, 00.1111 \text{ 1111}$$

$$+ [Y]_{\text{浮}} = 00 \text{ 111}, 00.1000 \text{ 0001}$$

$$[X+Y]_{\text{浮}} = 00 \text{ 111}, \underline{01}.1000 \text{ 0000}$$

对尾数(正的)运算结果进行右归,

尾数(连同符号位)右移1位, 阶码加1

$$[X+Y]_{\text{浮}} = 01 \text{ 000}, \underline{00}.1100 \text{ 0000} (0)$$

阶码溢出, 运算结果上溢。

目录：

1. 逻辑运算
2. 多功能算术/逻辑运算单元ALU。

逻辑运算

请自学“逻辑运算”这6页
(对位来计算)

- 1、逻辑非运算
- 2、逻辑加运算
- 3、逻辑乘运算
- 4、逻辑异或运算。

1、逻辑非运算

$X = x_0x_1x_2\dots x_n$, 对 x 求逻辑非, 则有

$$\overline{X} = Z = z_0z_1z_2\dots z_n$$

$$z_i = \overline{x_i}, i=0,1,2,\dots,n。$$

1、逻辑非运算

[例] $x_1=01001001$, $x_2=11110000$, 求 $\overline{x_1}$, $\overline{x_2}$

解:

$$\overline{x_1} = 10110110$$

$$\overline{x_2} = 00001111。$$

3、逻辑乘运算

[例] $x=10111001$, $y=11110011$, 求 $x \cdot y$

解:

$$\begin{array}{r} 10111001 \quad x \\ \cdot 11110011 \quad y \\ \hline 10110001 \quad z \end{array}$$

即 $x \cdot y = 10110001$ 。

4、逻辑异或运算

$$X = x_0x_1x_2\dots x_n, y = y_0y_1y_2\dots y_n$$

则有

$$x \oplus y = z = z_0z_1z_2\dots z_n$$

$$z_i = x_i \oplus y_i, i=0,1,2,\dots,n。$$

4、逻辑异或运算

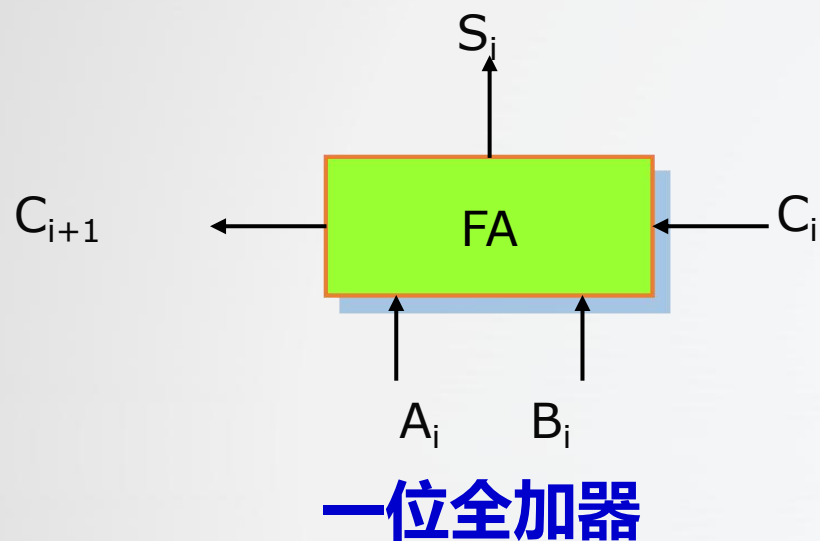
[例] $x=10101011$, $y=11001100$, 求 $x\oplus y$

解:

$$\begin{array}{r} 10101011 \quad x \\ \oplus 11001100 \quad y \\ \hline 01100111 \quad z \end{array}$$

即 $x\oplus y = 01100111$ 。

(回顾)带进位链的一位全加器



被加数 A_i	加数 B_i	低位进位 C_i	和 S_i	进位 C_{i+1}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

$$S_i = A_i \oplus B_i \oplus C_i$$

$$C_{i+1} = A_i B_i + A_i C_i + B_i C_i \quad (1)$$

对(1)变换可得:

$$C_{i+1} = A_i B_i + (A_i \oplus B_i) C_i$$

- 1位全加器FA的逻辑方程:

$$S_i = A_i \oplus B_i \oplus C_i$$

$$C_{i+1} = A_i B_i + (A_i \oplus B_i) C_i$$

令 $A_i \cdot B_i = G_i$

$A_i \oplus B_i = P_i$

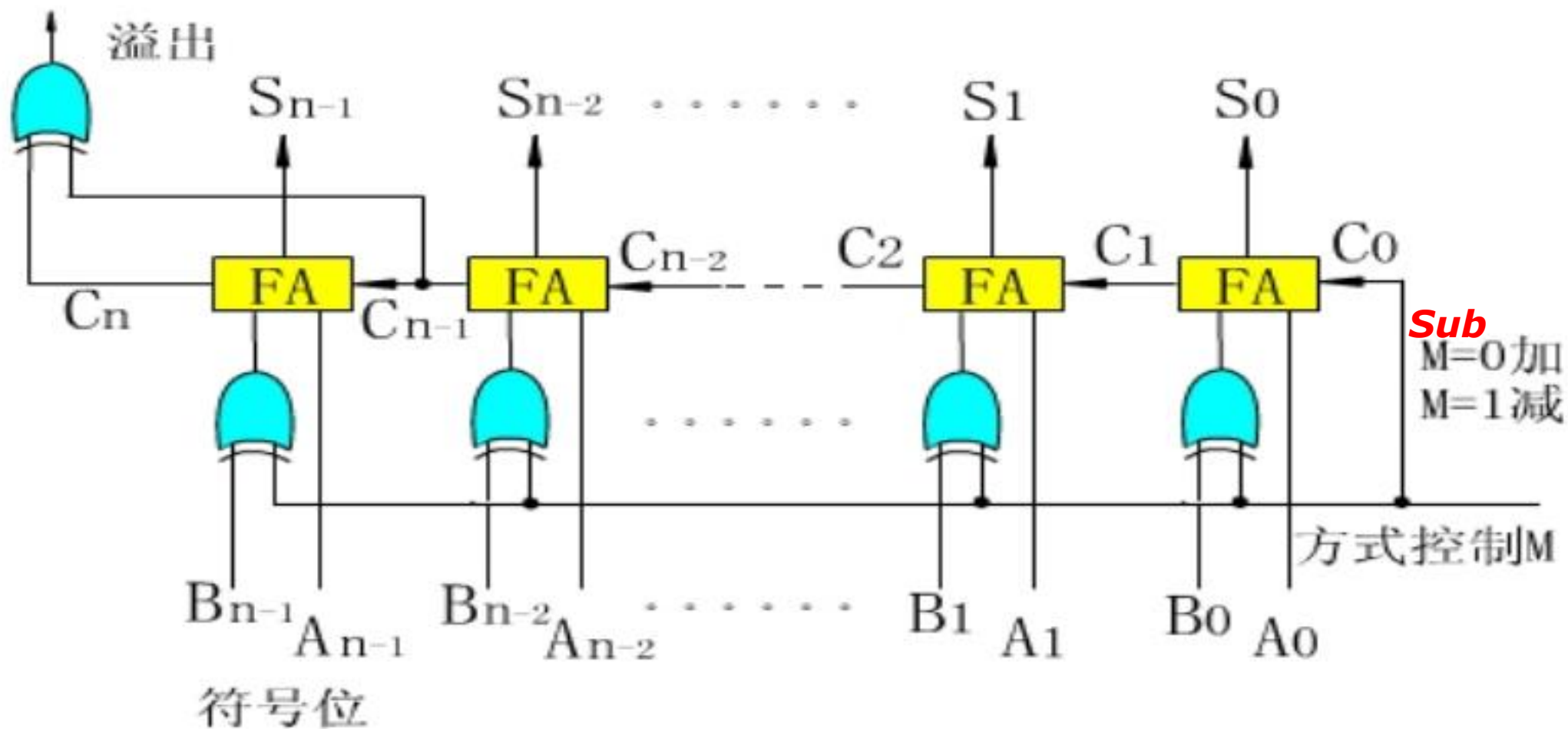
所以

$$C_{i+1} = G_i + P_i \cdot C_i$$

绝对进位

条件进位(传递进位)。

(回顾) n位串行进位加法器



串行进位的补码加法/减法器

串行进位(行波进位) 加法器特点

➤ 进位逻辑

特点: 进位信号逐位形成, 前后依赖, 因此速度较慢.

设n位加法器,

1) 逻辑式

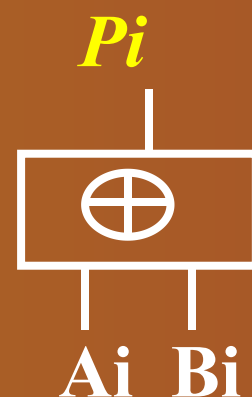
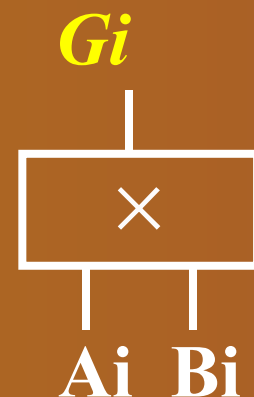
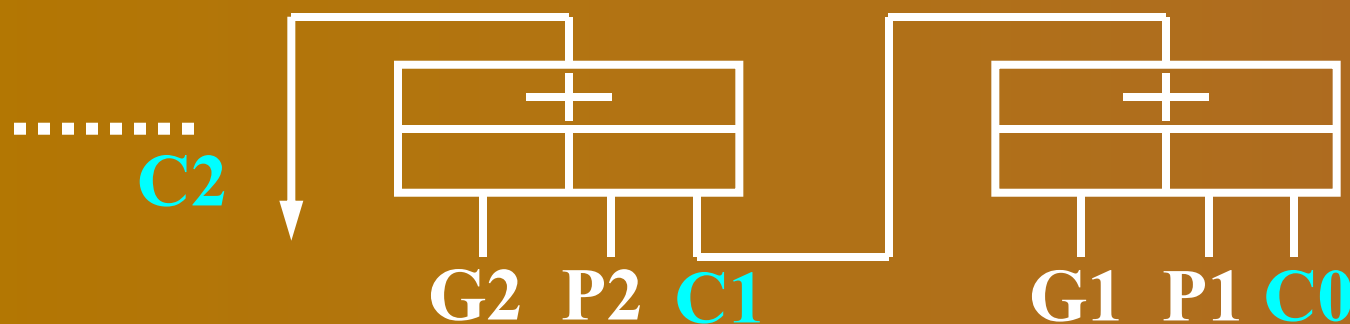
$$C_1 = G_1 + P_1 \underline{C_0}$$

$$C_2 = G_2 + P_2 \underline{C_1}$$

⋮

$$C_n = G_n + P_n \underline{C_{n-1}}$$

2) 结构举例





并行进位 加法器(重)

[特点] 各位进位信号 同时形成.

n位加法器, 进位逻辑如下:

1) 逻辑关系

$$C_1 = G_1 + P_1 C_0$$

$$C_2 = G_2 + P_2 C_1 \quad (\text{代换 } C_1)$$

$$= G_2 + P_2 G_1 + P_2 P_1 C_0$$

⋮

$$C_n = G_n + P_n C_{n-1} \quad (\text{代换 } C_{n-1})$$

$$= G_n + P_n G_{n-1} + \dots + P_n P_{n-1} \dots P_2 P_1 C_0$$

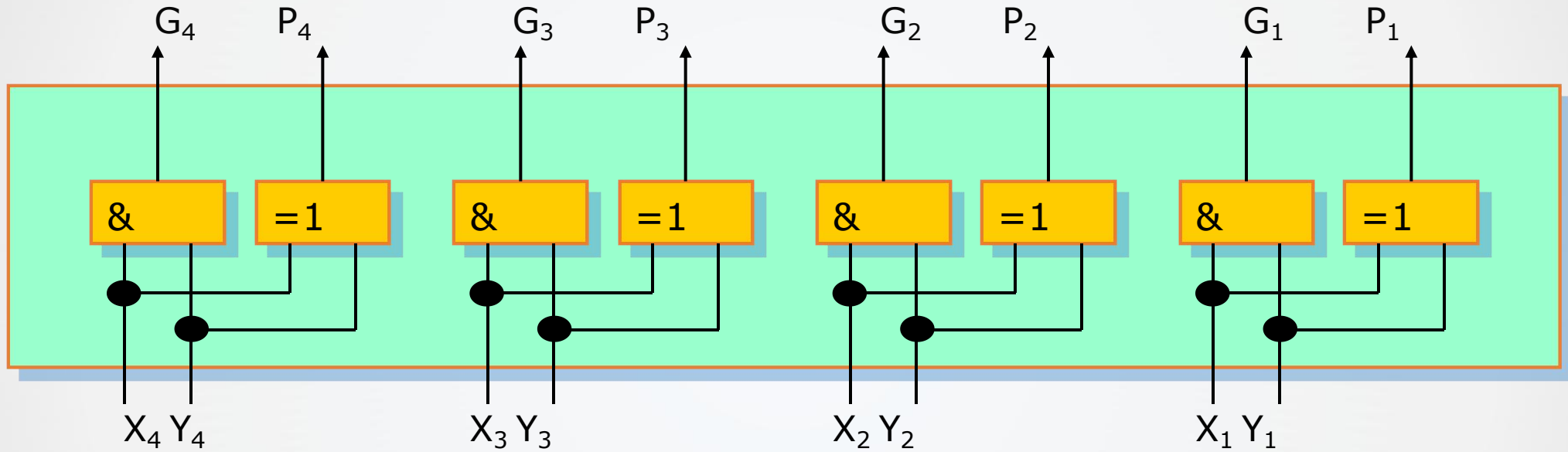
仅与最低位的进位输入 C_0 有关!

- ∴ 位数 n 越长, 进位链电路复杂度会越高
- ∴ 通常按照 4位一组 进行分组运算。

4位成组电路 (与门, 异或门)

$$X_i \cdot Y_i = G_i$$

$$X_i \oplus Y_i = P_i$$



■ $\therefore C_n = G_n + P_n G_{n-1} + \dots + P_n P_{n-1} \dots P_2 P_1 \cdot C_0$

■ 本位和 $S_1 = X_1 \oplus Y_1 \oplus C_0 = P_1 \oplus C_0$ (见下页电路图)

■ $S_2 = X_2 \oplus Y_2 \oplus C_1 = P_2 \oplus C_1$

■ $S_3 = X_3 \oplus Y_3 \oplus C_2 = P_3 \oplus C_2$

■ $S_4 = X_4 \oplus Y_4 \oplus C_3 = P_4 \oplus C_3$

■ $\therefore$ 最低位的进位信号 C_0 得到后, 所有 S_i 的运算只需要一级异或门即可完成 (见下页图)。

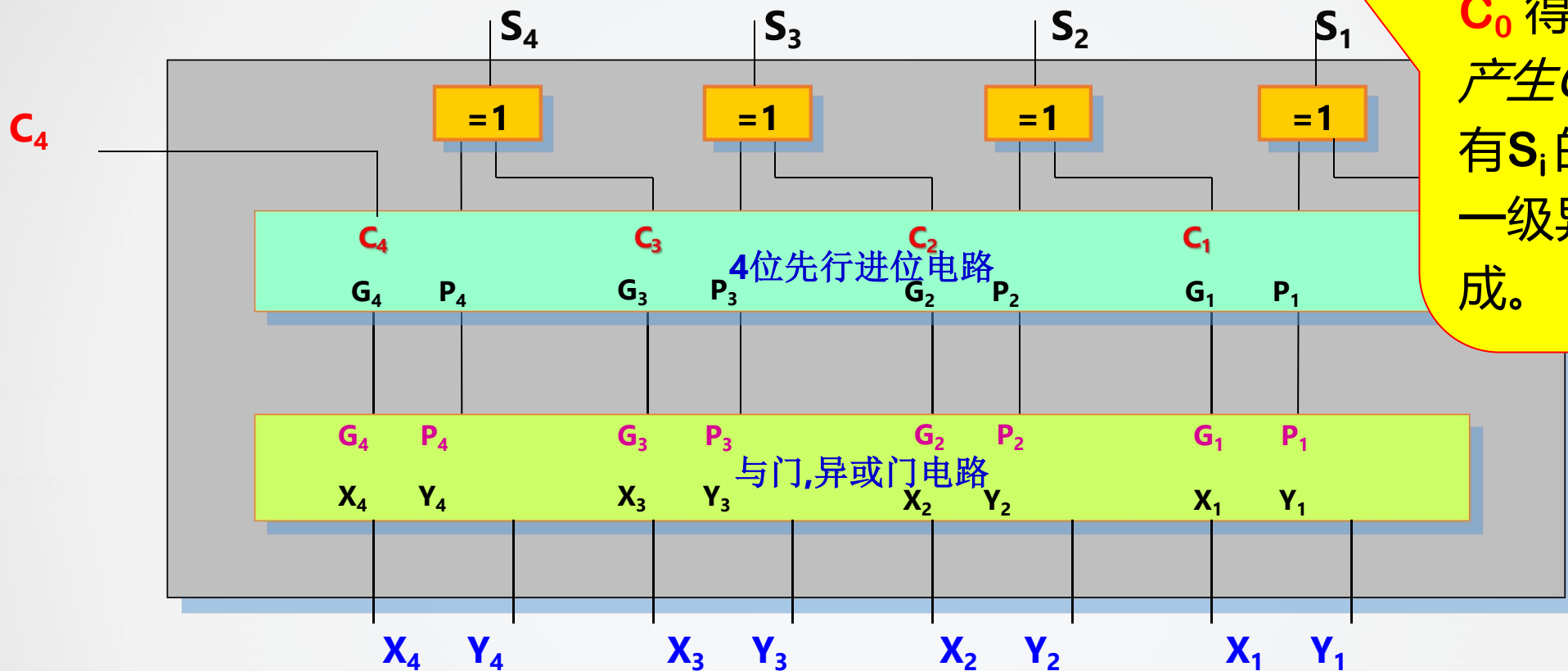
$$S_i = A_i \oplus B_i \oplus C_i$$

$$C_{i+1} = A_i B_i + A_i C_i + B_i C_i$$

$$= A_i B_i + (A_i \oplus B_i) C_i$$

令 $A_i \cdot B_i = G_i$
 $A_i \oplus B_i = P_i$

4位快速加法器 (并/先行进位)



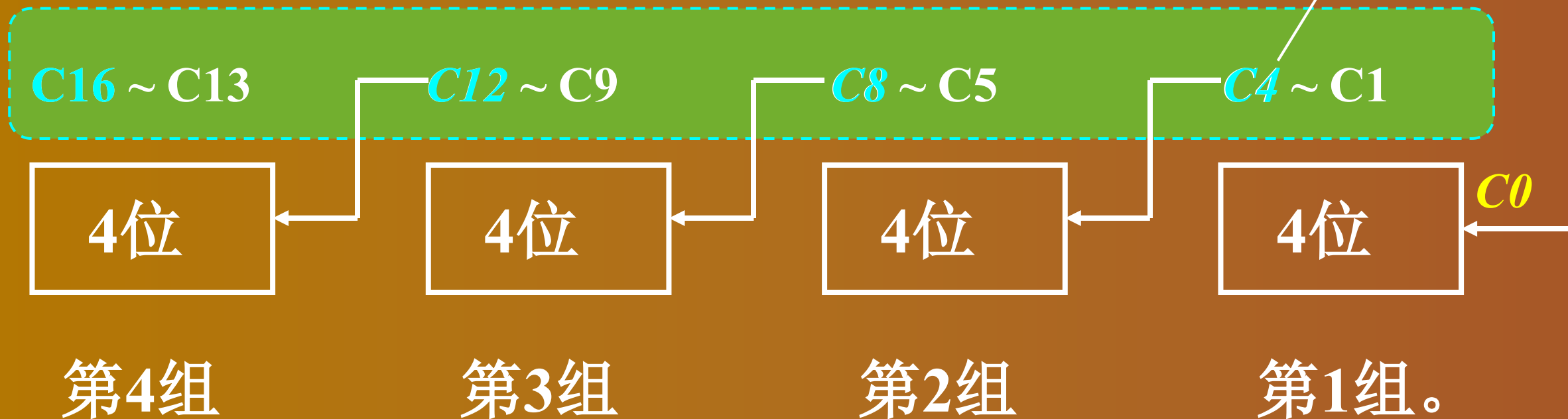
最低位的进位信号 C_0 得到后, 可并行产生 $C_1 \sim C_4$, 则所有 S_i 的运算只需要一级异或门即可完成。

$$\begin{aligned} S_1 &= X_1 \oplus Y_1 \oplus C_0 = P_1 \oplus C_0 \\ S_2 &= X_2 \oplus Y_2 \oplus C_1 = P_2 \oplus C_1 \\ S_3 &= X_3 \oplus Y_3 \oplus C_2 = P_3 \oplus C_2 \\ S_4 &= X_4 \oplus Y_4 \oplus C_3 = P_4 \oplus C_3 \end{aligned}$$

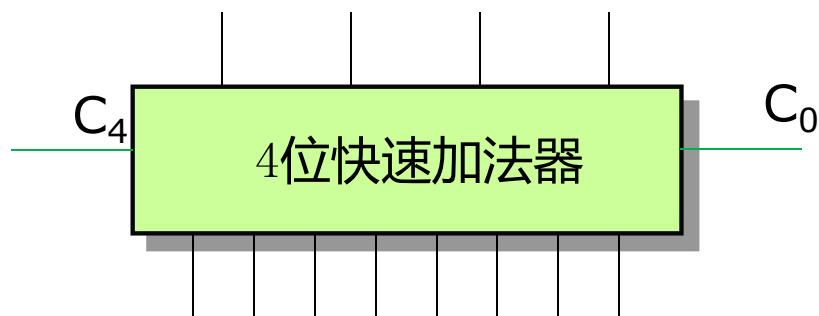
n位加法器的分组：组内并行、组间串行

设16位加法器，4位一组，分为4组：

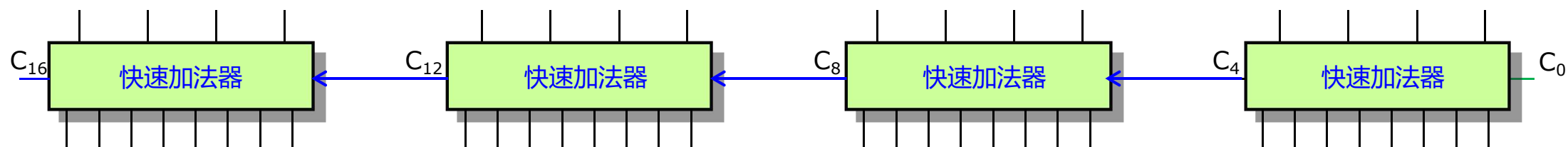
它是向下一组的最低位的进位输入



16位加法器



- 组内先行(并行)进位
- 组间 串行 进位
- 可否组间 并行? (芯片级联?)



成组进位

- $C_4 = G_4 + P_4 G_3 + P_4 P_3 G_2 + P_4 P_3 P_2 G_1 + P_4 P_3 P_2 P_1 C_0$
- 成组进位发生输出 $G^* = G_4 + P_4 G_3 + P_4 P_3 G_2 + P_4 P_3 P_2 G_1$
- 成组进位传递输出 $P^* = P_4 P_3 P_2 P_1$
- 其中 $G_i = A_i B_i$ $P_i = A_i \oplus B_i$
- 因此 $C_4 = G^* + P^* \cdot C_0$
- 对比以下形式:
- $C_1 = G_1 + P_1 C_0$.

多功能算术/逻辑运算单元ALU (重+难)

目录:

- 1、基本思想
- 2、逻辑表达式
- 3、算术逻辑运算的实现
- 4、两级先行进位的ALU。

1、基本思想

□ 目标:

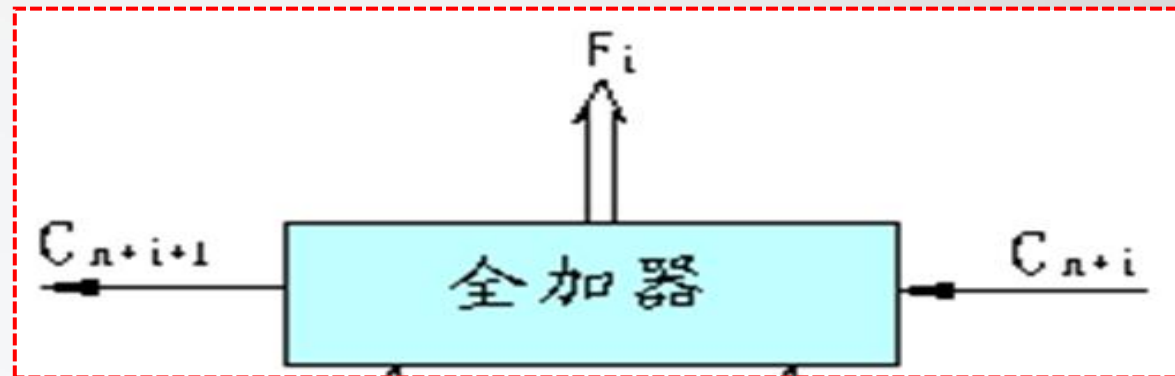
- (1) 实现并行(先行)进位
- (2) 实现16种算术运算, 16种逻辑运算.

□ 基本思想: 一位全加器FA的逻辑表达式:

$$F_i = A_i \oplus B_i \oplus C_{n+i}$$

$$C_{n+i+1} = A_i B_i + A_i C_{n+i} + B_i C_{n+i}$$

- 为了实现多种算术逻辑运算, 可将 A_i 和 B_i 输入一个函数发生器得到输出 X_i 和 Y_i , 作为一位全加器的输入 (见下页图)。



$$S_i = A_i \oplus B_i \oplus C_i$$

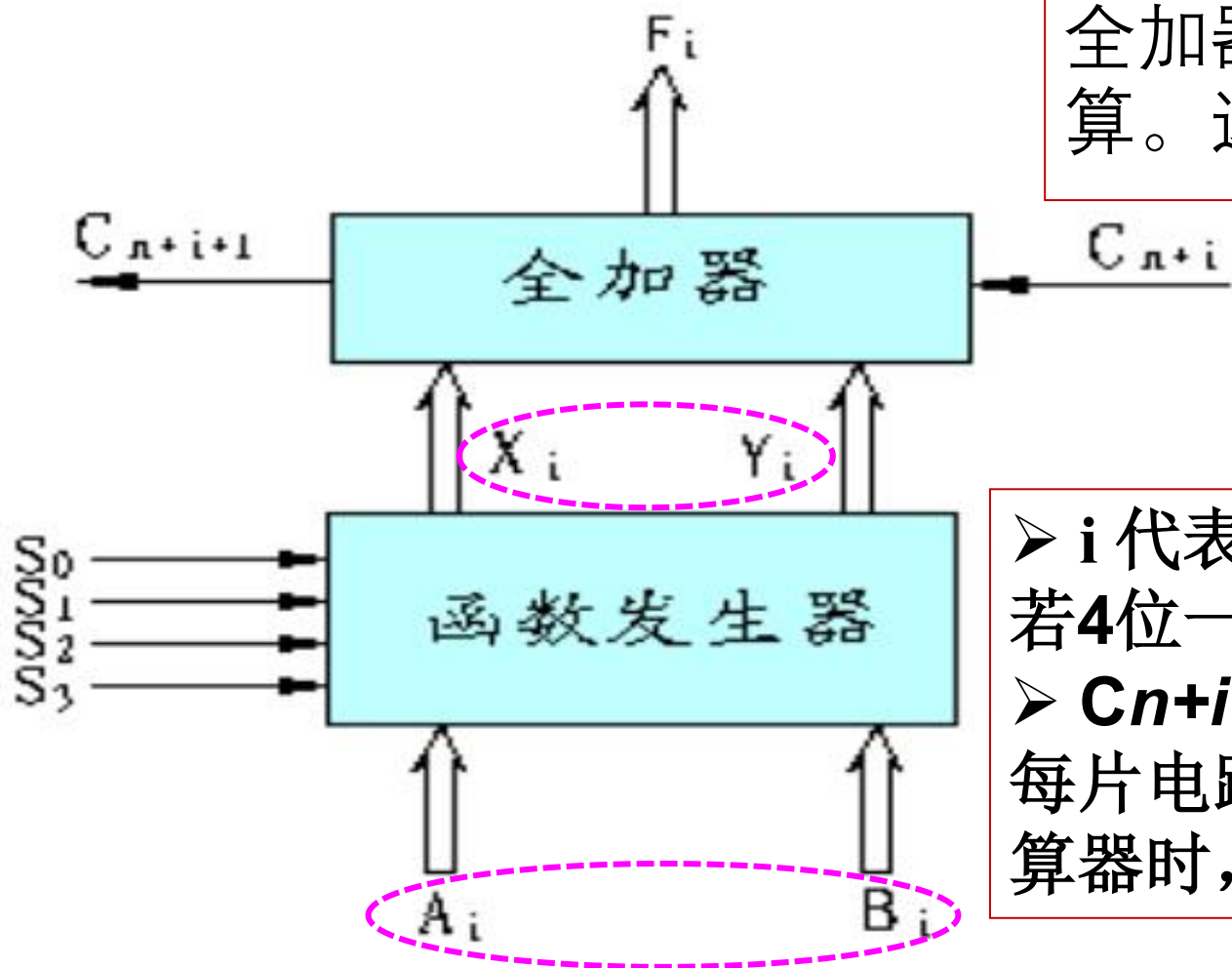
$$C_{i+1} = A_i B_i + A_i C_i + B_i C_i$$

$$C_{i+1} = A_i B_i + (A_i \oplus B_i) C_i$$



多功能算术逻辑单元

ALU的逻辑图与逻辑表达式:



- 将 A_i, B_i 先组合成由控制参数 S_0, S_1, S_2, S_3 控制的组合函数 X_i 和 Y_i , 再将 X_i, Y_i 和 C_{n+i} 通过全加器进行全加, 以实现多种算术、逻辑运算。这里:

$$F_i = X_i \oplus Y_i \oplus C_{n+i}$$

$$C_{n+i+1} = X_i Y_i + Y_i C_{n+i} + C_{n+i} X_i$$

- i 代表集成在一片电路上的ALU的二进制位数; 若4位一片, $i=0,1,2,3$
- C_{n+i} 代表若干片ALU组成更大字长运算器时每片电路的进位输入; 若当4片组成16位字长运算器时, $n=0,4,8,12$ 。

2、逻辑表达式

控制参数 S_0, S_1, S_2, S_3 控制输入 A_i 和 B_i , 产生 X_i 和 Y_i 的函数,如下表:

S_3	S_2	X_i	S_1	S_0	Y_i	S_3	S_2	X_i	S_1	S_0	Y_i
0	0	1	0	0	$\bar{A}_i$	1	0	$\bar{A}_i + \bar{B}_i$	1	0	$\bar{A}_i \cdot B_i$
0	1	$\bar{A}_i + B_i$	0	1	$\bar{A}_i \cdot \bar{B}_i$	1	1	$\bar{A}_i$	1	1	0

由此得到 X_i 和 Y_i 逻辑表达式

$$X_i = \overline{S_3 A_i B_i + S_2 A_i \bar{B}_i}$$

$$Y_i = \overline{A_i + S_0 B_i + S_1 \bar{B}_i}$$

又有: $C_{n+i+1} = X_i Y_i + Y_i C_{n+i} + C_{n+i} X_i$

综合这些式子可得:

$$C_{n+i+1} = Y_i + X_i C_{n+i}$$

若4位一片, $i = 0, 1, 2, 3$ 。

算术/逻辑运算单元ALU 74181

$$C_{n+i+1} = Y_i + X_i C_{n+i}$$

若4位一片， $i = 0, 1, 2, 3$ 。

- 由于每一位中 X_i 、 Y_i 产生是同时的,则可这样算出并行进位的 C_{n+4} :

$$C_{n+1} = Y_0 + X_0 C_n$$

$$C_{n+2} = Y_1 + X_1 C_{n+1} = Y_1 + Y_0 X_1 + X_0 X_1 C_n$$

$$C_{n+3} = Y_2 + X_2 C_{n+2} = Y_2 + Y_1 X_2 + Y_0 X_1 X_2 + X_0 X_1 X_2 C_n$$

$$C_{n+4} = Y_3 + X_3 C_{n+3} = Y_3 + Y_2 X_3 + Y_1 X_2 X_3 + Y_0 X_1 X_2 X_3 + X_0 X_1 X_2 X_3 C_n$$

- 令, 进位发生输出 $G = Y_3 + Y_2 X_3 + Y_1 X_2 X_3 + Y_0 X_1 X_2 X_3$

- 进位传递输出 $P = X_0 X_1 X_2 X_3$

- 则 $C_{n+4} = G + PC_n$ (1)

- 利用多片(组)74181和(1)式, 可实现组间的先行进位——先行进位发生器CLA, 器件为SN74182 (后述)。

3、SN 74181实现算术逻辑运算

- SN74181可实现16种逻辑运算和16种算术运算：

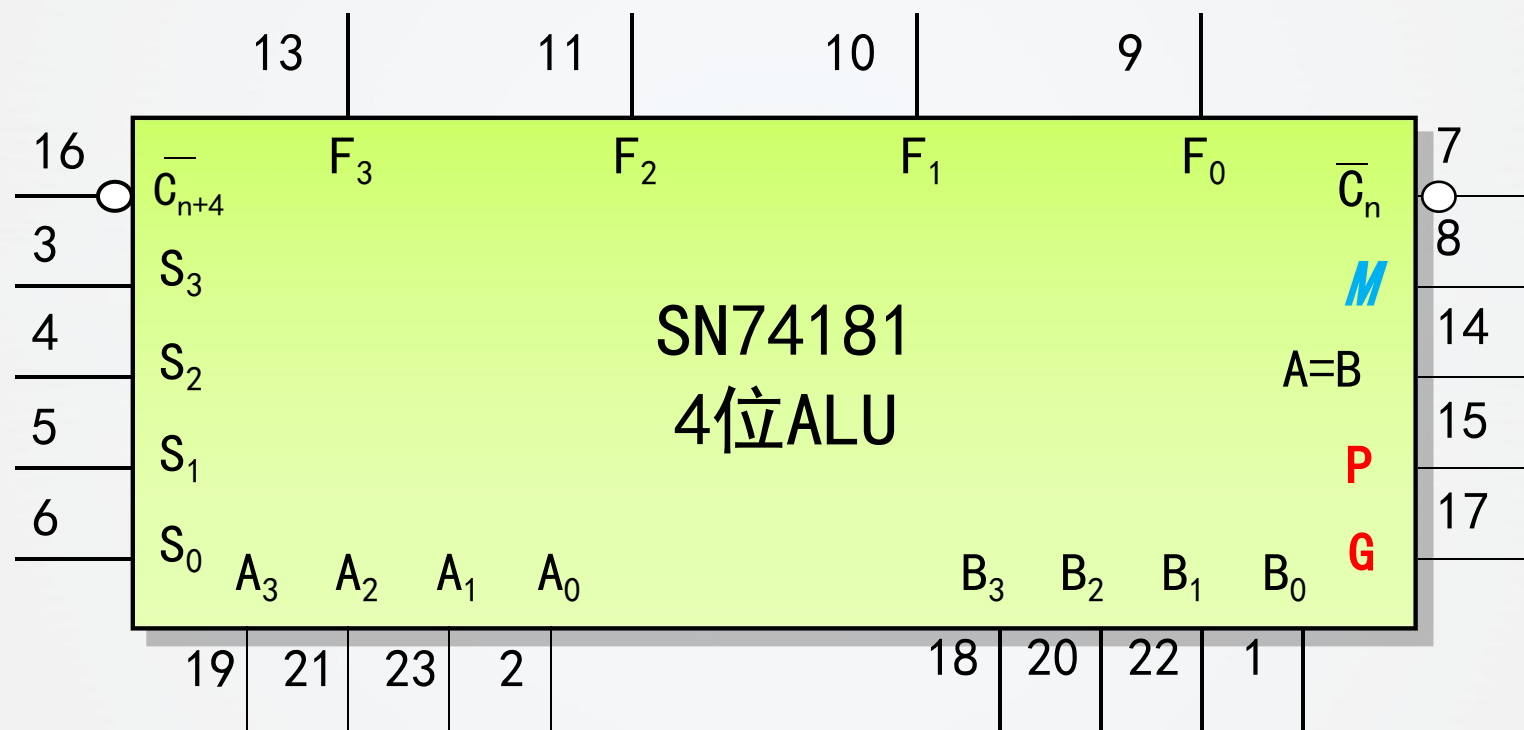
操作方式选择 $S_3 S_2 S_1 S_0$	逻辑运算 $M=H$	算术运算 $M=L, \bar{C}_n=H$
LLLL	$F=\bar{A}$	$F=A$
LLLH	$F=\overline{A+B}$	$F=A+B$
LLHL	$F=\bar{A}B$	$F=A+\bar{B}$
LLHH	$F=0$	$F=\text{减 } 1$
LHLL	$F=\overline{AB}$	$F=A \text{ 加 } \bar{A}\bar{B}$
LHLH	$F=\bar{B}$	$F=(A+B) \text{ 加 } \bar{A}\bar{B}$
LHHL	$F=A\oplus B$	$F=A \text{ 减 } B \text{ 减 } 1$
LHHH	$F=\bar{A}\bar{B}$	$F=\bar{A}\bar{B} \text{ 减 } 1$
HLLL	$F=\bar{A}+B$	$F=A \text{ 加 } AB$
HLLH	$F=\overline{A\oplus B}$	$F=A \text{ 加 } B$
HLHL	$F=B$	$F=(A+\bar{B}) \text{ 加 } AB$
HLHH	$F=AB$	$F=AB \text{ 减 } 1$
HHLL	$F=1$	$F=A \text{ 加 } A^*$
HHLH	$F=A+\bar{B}$	$F=(A+B) \text{ 加 } A$
HHHL	$F=A+B$	$F=(A+\bar{B}) \text{ 加 } A$
HHHH	$F=A$	$F=A \text{ 减 } 1$

- **M:** 运算方式选择符
- “加”“减”表示算术运算, +表示逻辑或。

SN74181引脚分配

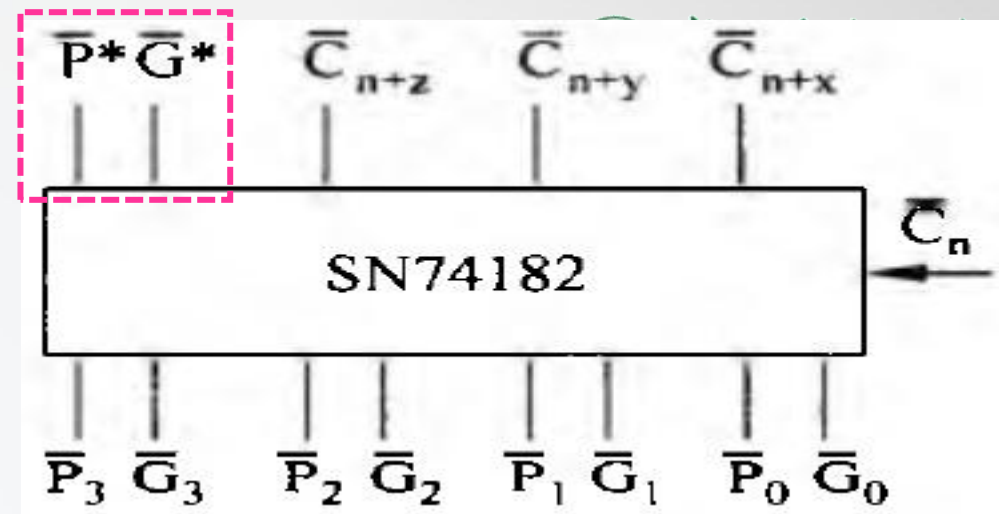
令, 进位发生输出 $G = Y_3 + Y_2X_3 + Y_1X_2X_3 + Y_0X_1X_2X_3$
进位传递输出 $P = X_0X_1X_2X_3$
则 $C_{n+4} = G + PC_n$ (1)

■ 4位 并行进位的多功能算术/逻辑运算单元的引脚分配图:



4.两级先行进位ALU 74182

设 4片74181输出依次为 P_0 、 $G_0 \sim P_3$ 、 G_3 ，连(输)入先行进位部件74182(右图。连入方式见下页)，得到如下进位逻辑：



- $C_{n+x} = G_0 + P_0 C_n$
- $C_{n+y} = G_1 + P_1 C_{n+x} = G_1 + G_0 P_1 + P_0 P_1 C_n$ (代入)
- $C_{n+z} = G_2 + P_2 C_{n+y} = G_2 + G_1 P_2 + G_0 P_1 P_2 + P_0 P_1 P_2 C_n$
- 最高位进位输出：
- $C_{n+w} = G_3 + P_3 C_{n+z}$
- $= G_3 + G_2 P_3 + G_1 P_2 P_3 + G_0 P_1 P_2 P_3 + P_0 P_1 P_2 P_3 C_n$
- $= G^* + P^* \cdot C_n$
- 其中，成组先行进位发生输出 $G^* = G_3 + G_2 P_3 + G_1 P_2 P_3 + G_0 P_1 P_2 P_3$
- ◆ 成组先行进位传递输出 $P^* = P_0 P_1 P_2 P_3$ 。

U

□ $C_{n+x} = G_0 + P_0 C_n$

□ $C_{n+y} = G_1 + P_1 C_{n+x} = G_1 + G_0 P_1 + P_0 P_1 C_n$

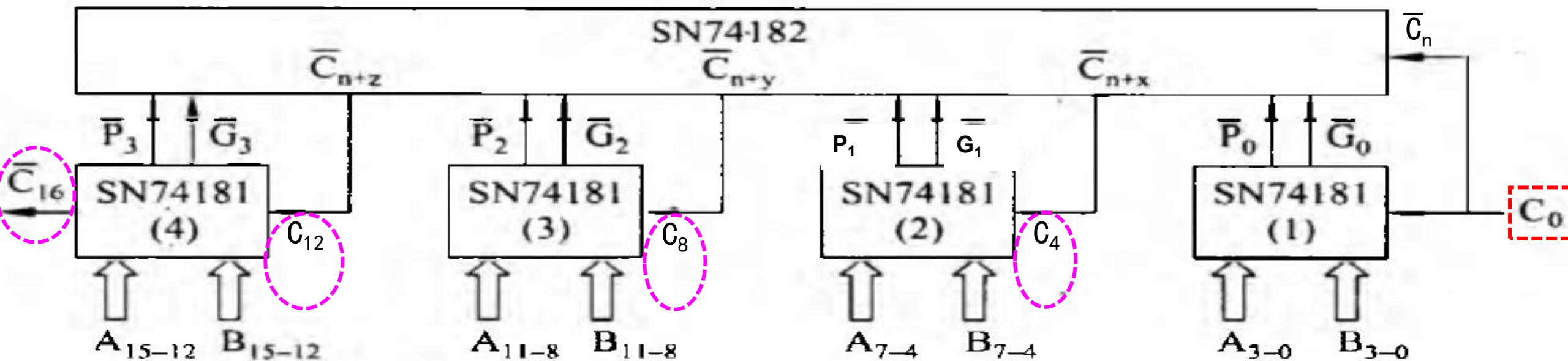
$$\square \quad C_{n+z} = G_2 + P_2 C_{n+y} = G_2 + G_1 P_2 + G_0 P_1 P_2 + P_0 P_1 P_2 C_n$$

$$\square \quad C_{n+w} = G^* + P^* \cdot C_n$$

$$\begin{aligned} X_i &= \frac{S_3 A_i B_i + S_2 A_i \overline{B_i}}{A_i + S_0 B_i + S_1 \overline{B_i}} \end{aligned}$$

进位发生输出 $G = Y_3 + Y_2X_3 + Y_1X_2X_3 + Y_0X_1X_2X_3$

进位传递输出 $P = X_0 X_1 X_2 X_3$



- 此16位ALU (组内先行进位, 组间先行进位):

$$P^* = P_0 P_1 P_2 P_3 \quad G^* = G_3 + G_2 P_3 + G_1 P_2 P_3 + G_0 P_1 P_2 P_3$$

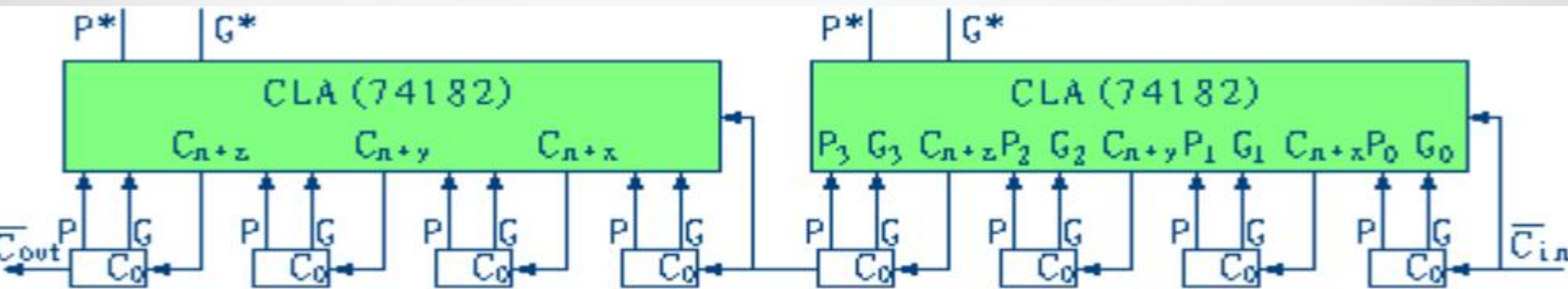
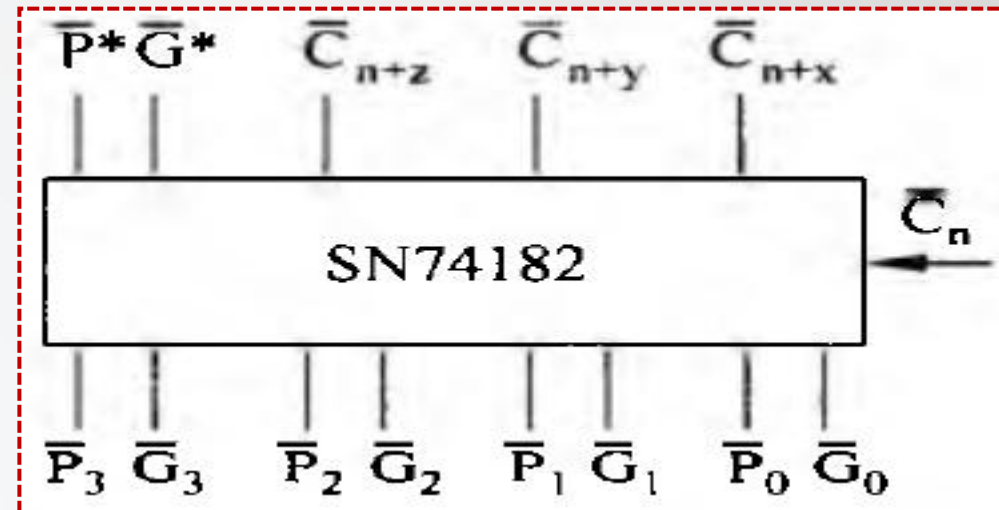
$$C_4 = \underline{G}_0 + \underline{P}_0 C_0 \quad C_8 = G_1 + P_1 G_0 + P_1 P_0 C_0 \quad C_{12} = \dots + \dots C_0 \quad C_{16} = G^* + P^* \cdot C_0$$

多功能算术/逻辑运算单元ALU

➤ 设计32位全并行ALU:

用 2片74182和

8片4位ALU74181组成.



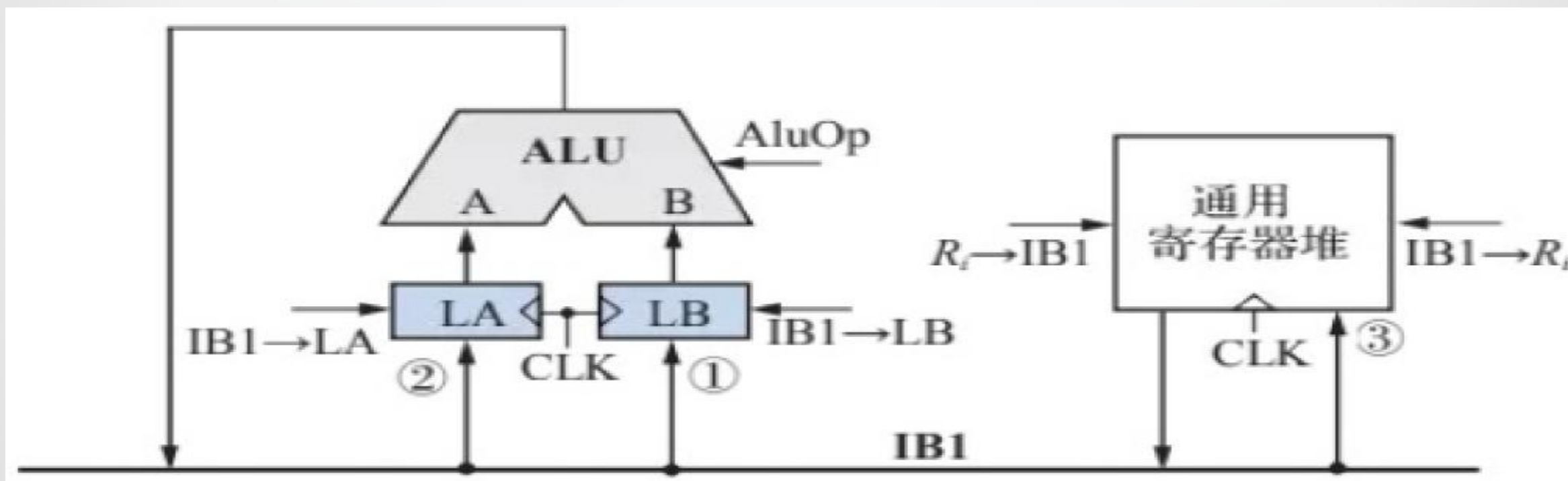
图：用两个16位全并行进位逻辑 级联组成的32位ALU

- 预备实验作业，本周日截止！
- 在本子上完成，需写出具体计算过程：
- 1、教材P93习题 3.8(1)：最后一步得到余数为负时，余数不移位，直接加上[Y]_补，得到正的余数。
- 2、教材习题3.9(2)；自学教材P85“IEEE754浮点数加减运算”后，完成习题3.10(1)
- 3、本子上完成试题书P27：设计题 4 (CLA: 进位部件)。
先独立完成，再对照答案订正。
- 4、在试题书上完成P7：2.1选择题：18,19,20,21,23。

内部总线

- 机器内部各部分数据传送频繁,可以把寄存器间的数据传送通路加以归并,组成总线结构,使不同来源的信息在此传输线上 分时传送 .
- 总线的分类:
 - **内部总线 (CPU内)**
 - **外部总线 (系统总线, 见《总线》章) 。**

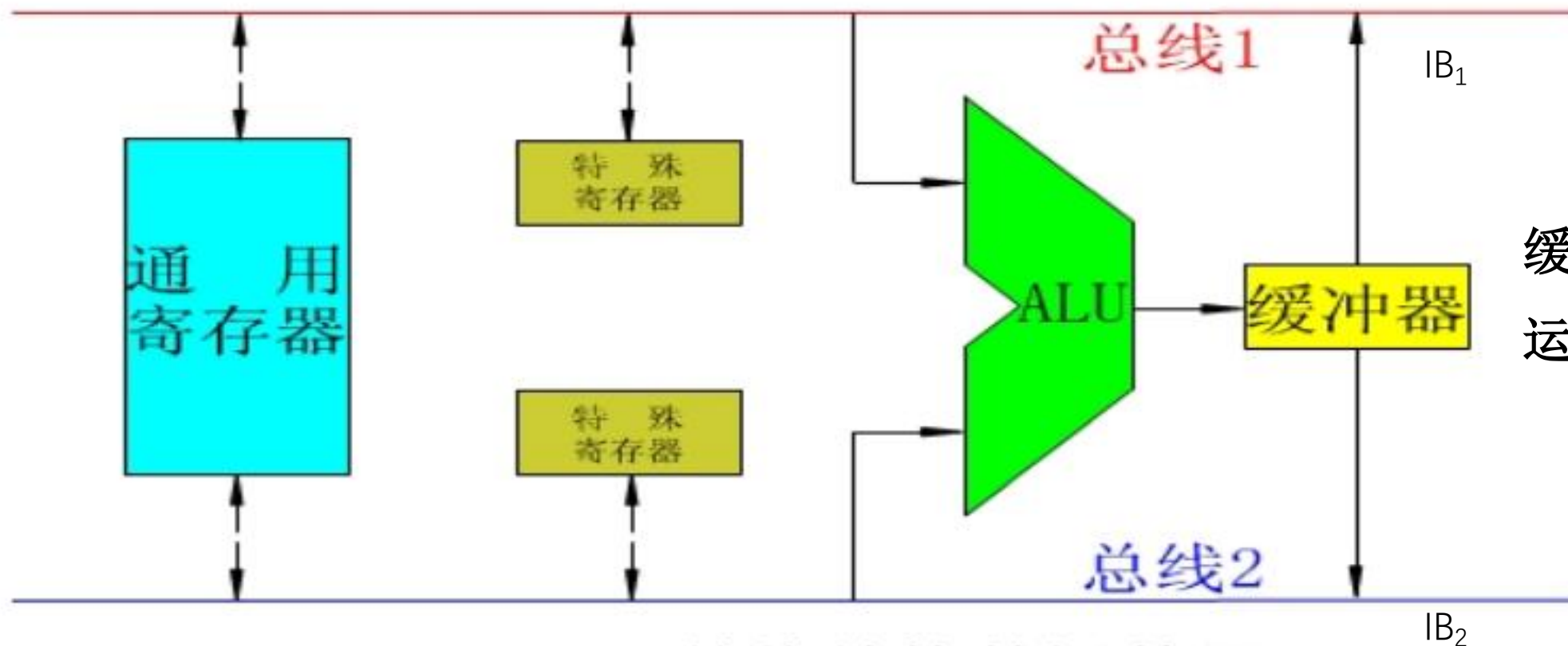
1、单总线结构的运算器



介绍详见教材P88。

2、双总线结构的运算器

请课下看教材自学
以下2页课件



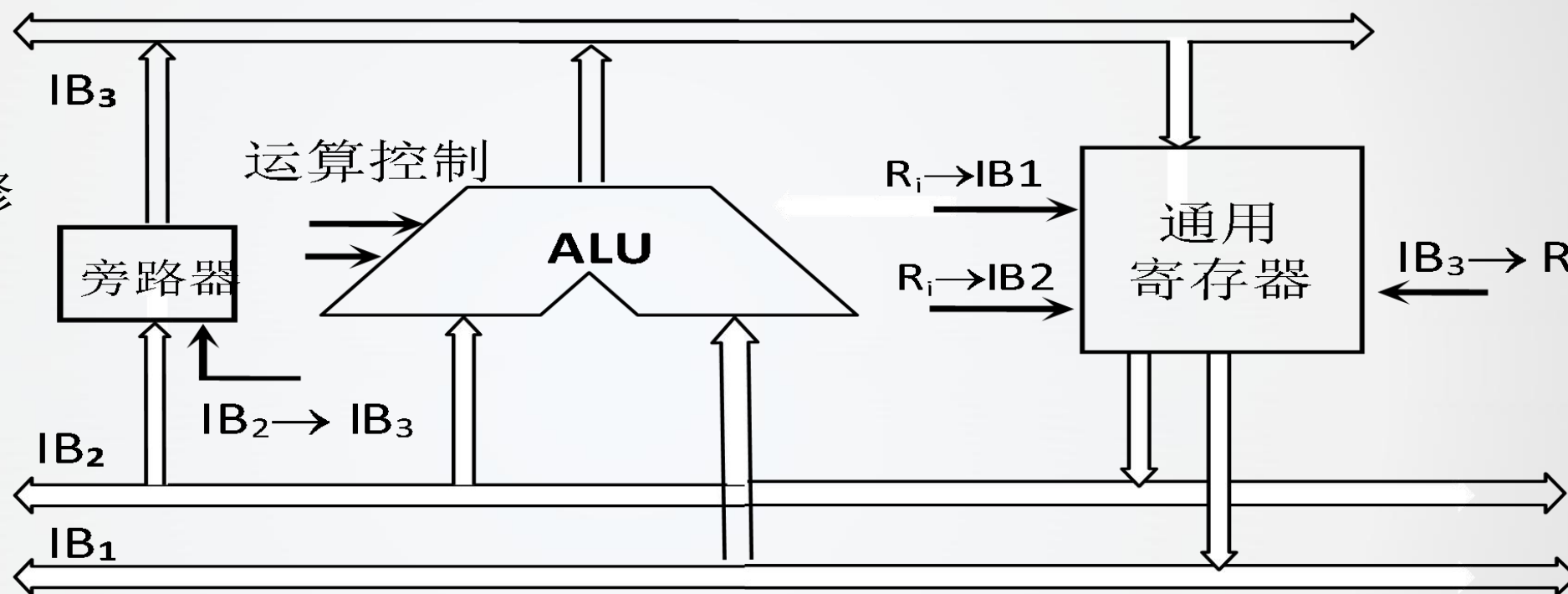
缓冲器：暂存
运算结果。

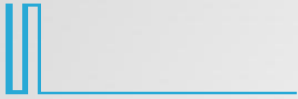
(b) 双总线结构的运算器

图2.15 运算器的三种基本结构形式

3、三总线结构的运算器

总线旁路器：作用是当操作数不需修改时，可以直接通过旁路器，从总线 IB_2 传输到 IB_3 。





The End

- 将数据进行**算术运算和逻辑运算**
- **暂存**参加运算的数据及运算的**中间结果**: 由 寄存器组
- **选取相应部件**中的数据参与运算: 由控制信号
- 反映运算处理的状态: **状态寄存器**。